4_rq1b_ml_models_rf_lstm_2

April 2, 2026

0.1 4_rq1b_ml_models_rf_lstm.ipynb

This code compares the performance of Random Forest and LSTM models for house price prediction by applying feature engineering, hyperparameter tuning, and cross-validation evaluation.

Research 1_b - Which ML model—Random Forest or Long Short-Term Memory (LSTM)—provides higher accuracy in predicting house prices?

0.1.1 1. Imports Library

[ ]:
```
import os
from pathlib import Path
import numpy as np
import pandas as pd
from sklearn.model_selection import KFold, TimeSeriesSplit, cross_val_score,␣
 ↪GridSearchCV, ParameterGrid
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score,␣
 ↪make_scorer
from sklearn.ensemble import RandomForestRegressor
from sklearn.impute import SimpleImputer
import tensorflow as tf
from tensorflow.keras import layers, models, callbacks, backend as K
from tensorflow.keras import regularizers
from sklearn.preprocessing import RobustScaler
import warnings
warnings.filterwarnings("ignore", category=Warning)

# Pretty printing
pd.set_option("display.float_format", lambda x: f"{x:,.4f}")
```

0.1.2 2. Load data

[ ]:
```
# Load data
data_path = Path("output/preprocess_csv/df_selected.csv")
df = pd.read_csv(data_path)
print(df.shape)
```

```
df.head(2)
```

```
(2919, 11)
```

[ ]:
```
      SalePrice  BedroomAbvGr  Bathrooms  PropertyAge  OverallQual  LotArea  \
0 208,500.0000             3     3.5000            5            7     8450
1 181,500.0000             3     2.5000           31            6     9600

   TotRmsAbvGrd  ScreenPorch  YrSold  MoSold QrtSold
0             8            0    2008       2      Q1
1             6            0    2007       5      Q2
```

0.1.3 3. Feature engineering

[ ]:
```
# Feature engineering

# Features set for ML
base_feats = [col for col in df.columns if col not in ['SalePrice',␣
 ↪'log_SalePrice']]

# Create log-transformed target variable if it doesn't exist
if "log_SalePrice" not in df.columns:
    df["log_SalePrice"] = np.log(df["SalePrice"])

# Copy dataframe
df_ml = df.copy()

# Display dataset information
print(f"ML dataset shape: {df_ml.shape}")
print(f"Number of features: {len(base_feats)}")
print(f"Sample size: {df_ml.shape[0]}")

# Check the prepared dataset
df_ml.head()
```

```
ML dataset shape: (2919, 12)
Number of features: 10
Sample size: 2919
```

[ ]:
```
      SalePrice  BedroomAbvGr  Bathrooms  PropertyAge  OverallQual  LotArea  \
0 208,500.0000             3     3.5000            5            7     8450
1 181,500.0000             3     2.5000           31            6     9600
2 223,500.0000             3     3.5000            7            7    11250
3 140,000.0000             3     2.0000           91            7     9550
4 250,000.0000             4     3.5000            8            8    14260

   TotRmsAbvGrd  ScreenPorch  YrSold  MoSold QrtSold  log_SalePrice
0             8            0    2008       2      Q1        12.2477
```

1	6	0	2007	5	Q2	12.1090
2	6	0	2008	9	Q3	12.3172
3	7	0	2006	2	Q1	11.8494
4	9	0	2008	12	Q4	12.4292

0.1.4 4. Evaluation Functions

[ ]:
```python
# Comprehensive evaluation metrics in PRICE space
def rmse_price(y_true_log, y_pred_log):
    """Root Mean Squared Error in price space ($)"""
    y_true = np.exp(y_true_log)
    y_pred = np.exp(y_pred_log)
    return np.sqrt(mean_squared_error(y_true, y_pred))

def mse_price(y_true_log, y_pred_log):
    """Mean Squared Error in price space ($²)"""
    y_true = np.exp(y_true_log)
    y_pred = np.exp(y_pred_log)
    return mean_squared_error(y_true, y_pred)

def mae_price(y_true_log, y_pred_log):
    """Mean Absolute Error in price space ($)"""
    y_true = np.exp(y_true_log)
    y_pred = np.exp(y_pred_log)
    return mean_absolute_error(y_true, y_pred)

def r2_price(y_true_log, y_pred_log):
    """R-squared in price space"""
    y_true = np.exp(y_true_log)
    y_pred = np.exp(y_pred_log)
    return r2_score(y_true, y_pred)

# Create scorers for cross-validation
rmse_scorer = make_scorer(rmse_price, greater_is_better=False)
mse_scorer = make_scorer(mse_price, greater_is_better=False)
mae_scorer = make_scorer(mae_price, greater_is_better=False)
r2_scorer = make_scorer(r2_price, greater_is_better=True)
```

0.1.5 5. Random Forest wit Hyperparameter Tuning

[ ]:
```python
# RANDOM FOREST - SIMPLIFIED VERSION

# Split numeric vs categorical
num_cols = [c for c in base_feats if df_ml[c].dtype != "O"]
cat_cols = [c for c in base_feats if df_ml[c].dtype == "O"]

X = df_ml[base_feats]
```

```
y = df_ml["log_SalePrice"].values

# Random Forest with Hyperparameter Tuning
print("\n" + "="*50)
print("RANDOM FOREST HYPERPARAMETER TUNING")
print("="*50)

# Preprocessing: Only encode categorical variables
print("Preprocessing data...")

# Preprocess categorical features once
cat_encoder = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
X_cat_encoded = cat_encoder.fit_transform(X[cat_cols])

# Combine numerical and categorical features
X_processed = np.hstack([X[num_cols].values, X_cat_encoded])
print(f"Processed feature matrix shape: {X_processed.shape}")

# Hyperparameter grid for Random Forest
rf_param_grid = {
    'n_estimators': [100, 300, 500],
    'max_depth': [10, 20, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'max_features': ['sqrt', 'log2', 0.8]
}

# Manual Grid Search with Cross-Validation
print("Performing Random Forest grid search...")
kf = KFold(n_splits=5, shuffle=True, random_state=42)

best_score = float('inf')
best_params = None
all_results = []

# Test all parameter combinations
total_combinations = (len(rf_param_grid['n_estimators']) *
                      len(rf_param_grid['max_depth']) *
                      len(rf_param_grid['min_samples_split']) *
                      len(rf_param_grid['min_samples_leaf']) *
                      len(rf_param_grid['max_features']))

combination_count = 0

for n_estimators in rf_param_grid['n_estimators']:
    for max_depth in rf_param_grid['max_depth']:
        for min_samples_split in rf_param_grid['min_samples_split']:
```

```
            for min_samples_leaf in rf_param_grid['min_samples_leaf']:
                for max_features in rf_param_grid['max_features']:

                    combination_count += 1
                    params = {
                        'n_estimators': n_estimators,
                        'max_depth': max_depth,
                        'min_samples_split': min_samples_split,
                        'min_samples_leaf': min_samples_leaf,
                        'max_features': max_features
                    }

                    print(f"Testing combination {combination_count}/
↪{total_combinations}: {params}")

                    rmse_scores, mse_scores, mae_scores, r2_scores = [], [],␣
↪[], []

                    # Cross-validation
                    for train_idx, val_idx in kf.split(X_processed):
                        X_train, X_val = X_processed[train_idx],␣
↪X_processed[val_idx]
                        y_train, y_val = y[train_idx], y[val_idx]

                        # Train Random Forest with current parameters
                        rf_model = RandomForestRegressor(
                            **params,
                            random_state=42,
                            n_jobs=-1
                        )
                        rf_model.fit(X_train, y_train)

                        # Predict and calculate metrics
                        y_pred = rf_model.predict(X_val)
                        rmse_scores.append(rmse_price(y_val, y_pred))
                        mse_scores.append(mse_price(y_val, y_pred))
                        mae_scores.append(mae_price(y_val, y_pred))
                        r2_scores.append(r2_price(y_val, y_pred))

                    # Calculate mean metrics across folds
                    mean_rmse = np.mean(rmse_scores)
                    std_rmse = np.std(rmse_scores)
                    mean_mse = np.mean(mse_scores)
                    mean_mae = np.mean(mae_scores)
                    mean_r2 = np.mean(r2_scores)
                    std_r2 = np.std(r2_scores)
```

```
                    all_results.append({
                        **params,
                        'mean_rmse': mean_rmse,
                        'std_rmse': std_rmse,
                        'mean_mse': mean_mse,
                        'mean_mae': mean_mae,
                        'mean_r2': mean_r2,
                        'std_r2': std_r2
                    })

                    # Update best parameters
                    if mean_rmse < best_score:
                        best_score = mean_rmse
                        best_params = params
                        print(f"   NEW BEST: RMSE = {mean_rmse:,.2f}")

# Convert results to DataFrame and sort
results_df = pd.DataFrame(all_results)
results_df = results_df.sort_values('mean_rmse')

print("\n" + "="*60)
print("RANDOM FOREST - FINAL RESULTS")
print("="*60)

print("\nBest Parameters:")
print(best_params)
print(f"Best RMSE: {best_score:,.2f}")

# Train final model with best parameters on full dataset
final_rf_model = RandomForestRegressor(**best_params, random_state=42,␣
 ↪n_jobs=-1)
final_rf_model.fit(X_processed, y)

# Get comprehensive metrics from best run
best_result = results_df.iloc[0]

print(f"\nRandom Forest - Comprehensive CV Performance:")
print(f"RMSE: {best_result['mean_rmse']:,.2f} (+/- {best_result['std_rmse']:,.
 ↪2f})")
print(f"MSE:  {best_result['mean_mse']:,.0f}")
print(f"MAE:  {best_result['mean_mae']:,.2f}")
print(f"R²   {best_result['mean_r2']:.4f} ± {best_result['std_r2']:.4f}")

# Store the benchmark for LSTM comparison
RF_BENCHMARK = best_result['mean_rmse']
print(f"\nRandom Forest Benchmark RMSE: {RF_BENCHMARK:,.2f}")
```

```
# Show top 5 parameter combinations
print("\nTop 5 Random Forest configurations:")
top_columns = ['n_estimators', 'max_depth', 'min_samples_split',␣
 ↪'min_samples_leaf', 'max_features', 'mean_rmse']
print(results_df[top_columns].head().round(2))

# Save results for comparison with LSTM
rf_results = {
    'best_params': best_params,
    'best_rmse': best_result['mean_rmse'],
    'best_rmse_std': best_result['std_rmse'],
    'best_mse': best_result['mean_mse'],
    'best_mae': best_result['mean_mae'],
    'best_r2': best_result['mean_r2'],
    'best_r2_std': best_result['std_r2'],
    'all_results': results_df,
    'model': final_rf_model,
    'feature_encoder': cat_encoder
}

print(f"\nRandom Forest tuning completed successfully!")
print(f"Total parameter combinations tested: {total_combinations}")
```

```
==================================================
RANDOM FOREST HYPERPARAMETER TUNING
==================================================
Preprocessing data…
Processed feature matrix shape: (2919, 13)
Performing Random Forest grid search…
Testing combination 1/243: {'n_estimators': 100, 'max_depth': 10,
'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 'sqrt'}
   NEW BEST: RMSE = 43,041.67
Testing combination 2/243: {'n_estimators': 100, 'max_depth': 10,
'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 'log2'}
Testing combination 3/243: {'n_estimators': 100, 'max_depth': 10,
'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 0.8}
Testing combination 4/243: {'n_estimators': 100, 'max_depth': 10,
'min_samples_split': 2, 'min_samples_leaf': 2, 'max_features': 'sqrt'}
Testing combination 5/243: {'n_estimators': 100, 'max_depth': 10,
'min_samples_split': 2, 'min_samples_leaf': 2, 'max_features': 'log2'}
Testing combination 6/243: {'n_estimators': 100, 'max_depth': 10,
'min_samples_split': 2, 'min_samples_leaf': 2, 'max_features': 0.8}
   NEW BEST: RMSE = 42,894.83
Testing combination 7/243: {'n_estimators': 100, 'max_depth': 10,
'min_samples_split': 2, 'min_samples_leaf': 4, 'max_features': 'sqrt'}
Testing combination 8/243: {'n_estimators': 100, 'max_depth': 10,
'min_samples_split': 2, 'min_samples_leaf': 4, 'max_features': 'log2'}
```

```
Testing combination 9/243: {'n_estimators': 100, 'max_depth': 10,
'min_samples_split': 2, 'min_samples_leaf': 4, 'max_features': 0.8}
Testing combination 10/243: {'n_estimators': 100, 'max_depth': 10,
'min_samples_split': 5, 'min_samples_leaf': 1, 'max_features': 'sqrt'}
Testing combination 11/243: {'n_estimators': 100, 'max_depth': 10,
'min_samples_split': 5, 'min_samples_leaf': 1, 'max_features': 'log2'}
Testing combination 12/243: {'n_estimators': 100, 'max_depth': 10,
'min_samples_split': 5, 'min_samples_leaf': 1, 'max_features': 0.8}
Testing combination 13/243: {'n_estimators': 100, 'max_depth': 10,
'min_samples_split': 5, 'min_samples_leaf': 2, 'max_features': 'sqrt'}
Testing combination 14/243: {'n_estimators': 100, 'max_depth': 10,
'min_samples_split': 5, 'min_samples_leaf': 2, 'max_features': 'log2'}
Testing combination 15/243: {'n_estimators': 100, 'max_depth': 10,
'min_samples_split': 5, 'min_samples_leaf': 2, 'max_features': 0.8}
Testing combination 16/243: {'n_estimators': 100, 'max_depth': 10,
'min_samples_split': 5, 'min_samples_leaf': 4, 'max_features': 'sqrt'}
Testing combination 17/243: {'n_estimators': 100, 'max_depth': 10,
'min_samples_split': 5, 'min_samples_leaf': 4, 'max_features': 'log2'}
Testing combination 18/243: {'n_estimators': 100, 'max_depth': 10,
'min_samples_split': 5, 'min_samples_leaf': 4, 'max_features': 0.8}
Testing combination 19/243: {'n_estimators': 100, 'max_depth': 10,
'min_samples_split': 10, 'min_samples_leaf': 1, 'max_features': 'sqrt'}
Testing combination 20/243: {'n_estimators': 100, 'max_depth': 10,
'min_samples_split': 10, 'min_samples_leaf': 1, 'max_features': 'log2'}
Testing combination 21/243: {'n_estimators': 100, 'max_depth': 10,
'min_samples_split': 10, 'min_samples_leaf': 1, 'max_features': 0.8}
Testing combination 22/243: {'n_estimators': 100, 'max_depth': 10,
'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': 'sqrt'}
Testing combination 23/243: {'n_estimators': 100, 'max_depth': 10,
'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': 'log2'}
Testing combination 24/243: {'n_estimators': 100, 'max_depth': 10,
'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': 0.8}
Testing combination 25/243: {'n_estimators': 100, 'max_depth': 10,
'min_samples_split': 10, 'min_samples_leaf': 4, 'max_features': 'sqrt'}
Testing combination 26/243: {'n_estimators': 100, 'max_depth': 10,
'min_samples_split': 10, 'min_samples_leaf': 4, 'max_features': 'log2'}
Testing combination 27/243: {'n_estimators': 100, 'max_depth': 10,
'min_samples_split': 10, 'min_samples_leaf': 4, 'max_features': 0.8}
Testing combination 28/243: {'n_estimators': 100, 'max_depth': 20,
'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 'sqrt'}
Testing combination 29/243: {'n_estimators': 100, 'max_depth': 20,
'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 'log2'}
Testing combination 30/243: {'n_estimators': 100, 'max_depth': 20,
'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 0.8}
Testing combination 31/243: {'n_estimators': 100, 'max_depth': 20,
'min_samples_split': 2, 'min_samples_leaf': 2, 'max_features': 'sqrt'}
Testing combination 32/243: {'n_estimators': 100, 'max_depth': 20,
'min_samples_split': 2, 'min_samples_leaf': 2, 'max_features': 'log2'}
```

```
Testing combination 33/243: {'n_estimators': 100, 'max_depth': 20,
'min_samples_split': 2, 'min_samples_leaf': 2, 'max_features': 0.8}
Testing combination 34/243: {'n_estimators': 100, 'max_depth': 20,
'min_samples_split': 2, 'min_samples_leaf': 4, 'max_features': 'sqrt'}
Testing combination 35/243: {'n_estimators': 100, 'max_depth': 20,
'min_samples_split': 2, 'min_samples_leaf': 4, 'max_features': 'log2'}
Testing combination 36/243: {'n_estimators': 100, 'max_depth': 20,
'min_samples_split': 2, 'min_samples_leaf': 4, 'max_features': 0.8}
Testing combination 37/243: {'n_estimators': 100, 'max_depth': 20,
'min_samples_split': 5, 'min_samples_leaf': 1, 'max_features': 'sqrt'}
Testing combination 38/243: {'n_estimators': 100, 'max_depth': 20,
'min_samples_split': 5, 'min_samples_leaf': 1, 'max_features': 'log2'}
Testing combination 39/243: {'n_estimators': 100, 'max_depth': 20,
'min_samples_split': 5, 'min_samples_leaf': 1, 'max_features': 0.8}
Testing combination 40/243: {'n_estimators': 100, 'max_depth': 20,
'min_samples_split': 5, 'min_samples_leaf': 2, 'max_features': 'sqrt'}
Testing combination 41/243: {'n_estimators': 100, 'max_depth': 20,
'min_samples_split': 5, 'min_samples_leaf': 2, 'max_features': 'log2'}
Testing combination 42/243: {'n_estimators': 100, 'max_depth': 20,
'min_samples_split': 5, 'min_samples_leaf': 2, 'max_features': 0.8}
Testing combination 43/243: {'n_estimators': 100, 'max_depth': 20,
'min_samples_split': 5, 'min_samples_leaf': 4, 'max_features': 'sqrt'}
Testing combination 44/243: {'n_estimators': 100, 'max_depth': 20,
'min_samples_split': 5, 'min_samples_leaf': 4, 'max_features': 'log2'}
Testing combination 45/243: {'n_estimators': 100, 'max_depth': 20,
'min_samples_split': 5, 'min_samples_leaf': 4, 'max_features': 0.8}
Testing combination 46/243: {'n_estimators': 100, 'max_depth': 20,
'min_samples_split': 10, 'min_samples_leaf': 1, 'max_features': 'sqrt'}
   NEW BEST: RMSE = 42,835.79
Testing combination 47/243: {'n_estimators': 100, 'max_depth': 20,
'min_samples_split': 10, 'min_samples_leaf': 1, 'max_features': 'log2'}
   NEW BEST: RMSE = 42,835.79
Testing combination 48/243: {'n_estimators': 100, 'max_depth': 20,
'min_samples_split': 10, 'min_samples_leaf': 1, 'max_features': 0.8}
Testing combination 49/243: {'n_estimators': 100, 'max_depth': 20,
'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': 'sqrt'}
Testing combination 50/243: {'n_estimators': 100, 'max_depth': 20,
'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': 'log2'}
Testing combination 51/243: {'n_estimators': 100, 'max_depth': 20,
'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': 0.8}
Testing combination 52/243: {'n_estimators': 100, 'max_depth': 20,
'min_samples_split': 10, 'min_samples_leaf': 4, 'max_features': 'sqrt'}
Testing combination 53/243: {'n_estimators': 100, 'max_depth': 20,
'min_samples_split': 10, 'min_samples_leaf': 4, 'max_features': 'log2'}
Testing combination 54/243: {'n_estimators': 100, 'max_depth': 20,
'min_samples_split': 10, 'min_samples_leaf': 4, 'max_features': 0.8}
Testing combination 55/243: {'n_estimators': 100, 'max_depth': None,
'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 'sqrt'}
```

```
Testing combination 56/243: {'n_estimators': 100, 'max_depth': None,
'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 'log2'}
Testing combination 57/243: {'n_estimators': 100, 'max_depth': None,
'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 0.8}
Testing combination 58/243: {'n_estimators': 100, 'max_depth': None,
'min_samples_split': 2, 'min_samples_leaf': 2, 'max_features': 'sqrt'}
Testing combination 59/243: {'n_estimators': 100, 'max_depth': None,
'min_samples_split': 2, 'min_samples_leaf': 2, 'max_features': 'log2'}
Testing combination 60/243: {'n_estimators': 100, 'max_depth': None,
'min_samples_split': 2, 'min_samples_leaf': 2, 'max_features': 0.8}
Testing combination 61/243: {'n_estimators': 100, 'max_depth': None,
'min_samples_split': 2, 'min_samples_leaf': 4, 'max_features': 'sqrt'}
Testing combination 62/243: {'n_estimators': 100, 'max_depth': None,
'min_samples_split': 2, 'min_samples_leaf': 4, 'max_features': 'log2'}
Testing combination 63/243: {'n_estimators': 100, 'max_depth': None,
'min_samples_split': 2, 'min_samples_leaf': 4, 'max_features': 0.8}
Testing combination 64/243: {'n_estimators': 100, 'max_depth': None,
'min_samples_split': 5, 'min_samples_leaf': 1, 'max_features': 'sqrt'}
Testing combination 65/243: {'n_estimators': 100, 'max_depth': None,
'min_samples_split': 5, 'min_samples_leaf': 1, 'max_features': 'log2'}
Testing combination 66/243: {'n_estimators': 100, 'max_depth': None,
'min_samples_split': 5, 'min_samples_leaf': 1, 'max_features': 0.8}
Testing combination 67/243: {'n_estimators': 100, 'max_depth': None,
'min_samples_split': 5, 'min_samples_leaf': 2, 'max_features': 'sqrt'}
Testing combination 68/243: {'n_estimators': 100, 'max_depth': None,
'min_samples_split': 5, 'min_samples_leaf': 2, 'max_features': 'log2'}
Testing combination 69/243: {'n_estimators': 100, 'max_depth': None,
'min_samples_split': 5, 'min_samples_leaf': 2, 'max_features': 0.8}
Testing combination 70/243: {'n_estimators': 100, 'max_depth': None,
'min_samples_split': 5, 'min_samples_leaf': 4, 'max_features': 'sqrt'}
Testing combination 71/243: {'n_estimators': 100, 'max_depth': None,
'min_samples_split': 5, 'min_samples_leaf': 4, 'max_features': 'log2'}
Testing combination 72/243: {'n_estimators': 100, 'max_depth': None,
'min_samples_split': 5, 'min_samples_leaf': 4, 'max_features': 0.8}
Testing combination 73/243: {'n_estimators': 100, 'max_depth': None,
'min_samples_split': 10, 'min_samples_leaf': 1, 'max_features': 'sqrt'}
Testing combination 74/243: {'n_estimators': 100, 'max_depth': None,
'min_samples_split': 10, 'min_samples_leaf': 1, 'max_features': 'log2'}
Testing combination 75/243: {'n_estimators': 100, 'max_depth': None,
'min_samples_split': 10, 'min_samples_leaf': 1, 'max_features': 0.8}
Testing combination 76/243: {'n_estimators': 100, 'max_depth': None,
'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': 'sqrt'}
Testing combination 77/243: {'n_estimators': 100, 'max_depth': None,
'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': 'log2'}
Testing combination 78/243: {'n_estimators': 100, 'max_depth': None,
'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': 0.8}
Testing combination 79/243: {'n_estimators': 100, 'max_depth': None,
'min_samples_split': 10, 'min_samples_leaf': 4, 'max_features': 'sqrt'}
```

```
Testing combination 80/243: {'n_estimators': 100, 'max_depth': None,
'min_samples_split': 10, 'min_samples_leaf': 4, 'max_features': 'log2'}
Testing combination 81/243: {'n_estimators': 100, 'max_depth': None,
'min_samples_split': 10, 'min_samples_leaf': 4, 'max_features': 0.8}
Testing combination 82/243: {'n_estimators': 300, 'max_depth': 10,
'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 'sqrt'}
Testing combination 83/243: {'n_estimators': 300, 'max_depth': 10,
'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 'log2'}
Testing combination 84/243: {'n_estimators': 300, 'max_depth': 10,
'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 0.8}
Testing combination 85/243: {'n_estimators': 300, 'max_depth': 10,
'min_samples_split': 2, 'min_samples_leaf': 2, 'max_features': 'sqrt'}
Testing combination 86/243: {'n_estimators': 300, 'max_depth': 10,
'min_samples_split': 2, 'min_samples_leaf': 2, 'max_features': 'log2'}
Testing combination 87/243: {'n_estimators': 300, 'max_depth': 10,
'min_samples_split': 2, 'min_samples_leaf': 2, 'max_features': 0.8}
Testing combination 88/243: {'n_estimators': 300, 'max_depth': 10,
'min_samples_split': 2, 'min_samples_leaf': 4, 'max_features': 'sqrt'}
Testing combination 89/243: {'n_estimators': 300, 'max_depth': 10,
'min_samples_split': 2, 'min_samples_leaf': 4, 'max_features': 'log2'}
Testing combination 90/243: {'n_estimators': 300, 'max_depth': 10,
'min_samples_split': 2, 'min_samples_leaf': 4, 'max_features': 0.8}
Testing combination 91/243: {'n_estimators': 300, 'max_depth': 10,
'min_samples_split': 5, 'min_samples_leaf': 1, 'max_features': 'sqrt'}
Testing combination 92/243: {'n_estimators': 300, 'max_depth': 10,
'min_samples_split': 5, 'min_samples_leaf': 1, 'max_features': 'log2'}
Testing combination 93/243: {'n_estimators': 300, 'max_depth': 10,
'min_samples_split': 5, 'min_samples_leaf': 1, 'max_features': 0.8}
Testing combination 94/243: {'n_estimators': 300, 'max_depth': 10,
'min_samples_split': 5, 'min_samples_leaf': 2, 'max_features': 'sqrt'}
Testing combination 95/243: {'n_estimators': 300, 'max_depth': 10,
'min_samples_split': 5, 'min_samples_leaf': 2, 'max_features': 'log2'}
Testing combination 96/243: {'n_estimators': 300, 'max_depth': 10,
'min_samples_split': 5, 'min_samples_leaf': 2, 'max_features': 0.8}
Testing combination 97/243: {'n_estimators': 300, 'max_depth': 10,
'min_samples_split': 5, 'min_samples_leaf': 4, 'max_features': 'sqrt'}
Testing combination 98/243: {'n_estimators': 300, 'max_depth': 10,
'min_samples_split': 5, 'min_samples_leaf': 4, 'max_features': 'log2'}
Testing combination 99/243: {'n_estimators': 300, 'max_depth': 10,
'min_samples_split': 5, 'min_samples_leaf': 4, 'max_features': 0.8}
Testing combination 100/243: {'n_estimators': 300, 'max_depth': 10,
'min_samples_split': 10, 'min_samples_leaf': 1, 'max_features': 'sqrt'}
Testing combination 101/243: {'n_estimators': 300, 'max_depth': 10,
'min_samples_split': 10, 'min_samples_leaf': 1, 'max_features': 'log2'}
Testing combination 102/243: {'n_estimators': 300, 'max_depth': 10,
'min_samples_split': 10, 'min_samples_leaf': 1, 'max_features': 0.8}
Testing combination 103/243: {'n_estimators': 300, 'max_depth': 10,
'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': 'sqrt'}
```

```
Testing combination 104/243: {'n_estimators': 300, 'max_depth': 10,
'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': 'log2'}
Testing combination 105/243: {'n_estimators': 300, 'max_depth': 10,
'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': 0.8}
Testing combination 106/243: {'n_estimators': 300, 'max_depth': 10,
'min_samples_split': 10, 'min_samples_leaf': 4, 'max_features': 'sqrt'}
Testing combination 107/243: {'n_estimators': 300, 'max_depth': 10,
'min_samples_split': 10, 'min_samples_leaf': 4, 'max_features': 'log2'}
Testing combination 108/243: {'n_estimators': 300, 'max_depth': 10,
'min_samples_split': 10, 'min_samples_leaf': 4, 'max_features': 0.8}
Testing combination 109/243: {'n_estimators': 300, 'max_depth': 20,
'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 'sqrt'}
Testing combination 110/243: {'n_estimators': 300, 'max_depth': 20,
'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 'log2'}
Testing combination 111/243: {'n_estimators': 300, 'max_depth': 20,
'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 0.8}
Testing combination 112/243: {'n_estimators': 300, 'max_depth': 20,
'min_samples_split': 2, 'min_samples_leaf': 2, 'max_features': 'sqrt'}
Testing combination 113/243: {'n_estimators': 300, 'max_depth': 20,
'min_samples_split': 2, 'min_samples_leaf': 2, 'max_features': 'log2'}
Testing combination 114/243: {'n_estimators': 300, 'max_depth': 20,
'min_samples_split': 2, 'min_samples_leaf': 2, 'max_features': 0.8}
Testing combination 115/243: {'n_estimators': 300, 'max_depth': 20,
'min_samples_split': 2, 'min_samples_leaf': 4, 'max_features': 'sqrt'}
Testing combination 116/243: {'n_estimators': 300, 'max_depth': 20,
'min_samples_split': 2, 'min_samples_leaf': 4, 'max_features': 'log2'}
Testing combination 117/243: {'n_estimators': 300, 'max_depth': 20,
'min_samples_split': 2, 'min_samples_leaf': 4, 'max_features': 0.8}
Testing combination 118/243: {'n_estimators': 300, 'max_depth': 20,
'min_samples_split': 5, 'min_samples_leaf': 1, 'max_features': 'sqrt'}
Testing combination 119/243: {'n_estimators': 300, 'max_depth': 20,
'min_samples_split': 5, 'min_samples_leaf': 1, 'max_features': 'log2'}
Testing combination 120/243: {'n_estimators': 300, 'max_depth': 20,
'min_samples_split': 5, 'min_samples_leaf': 1, 'max_features': 0.8}
Testing combination 121/243: {'n_estimators': 300, 'max_depth': 20,
'min_samples_split': 5, 'min_samples_leaf': 2, 'max_features': 'sqrt'}
Testing combination 122/243: {'n_estimators': 300, 'max_depth': 20,
'min_samples_split': 5, 'min_samples_leaf': 2, 'max_features': 'log2'}
Testing combination 123/243: {'n_estimators': 300, 'max_depth': 20,
'min_samples_split': 5, 'min_samples_leaf': 2, 'max_features': 0.8}
Testing combination 124/243: {'n_estimators': 300, 'max_depth': 20,
'min_samples_split': 5, 'min_samples_leaf': 4, 'max_features': 'sqrt'}
Testing combination 125/243: {'n_estimators': 300, 'max_depth': 20,
'min_samples_split': 5, 'min_samples_leaf': 4, 'max_features': 'log2'}
Testing combination 126/243: {'n_estimators': 300, 'max_depth': 20,
'min_samples_split': 5, 'min_samples_leaf': 4, 'max_features': 0.8}
Testing combination 127/243: {'n_estimators': 300, 'max_depth': 20,
'min_samples_split': 10, 'min_samples_leaf': 1, 'max_features': 'sqrt'}
```

```
Testing combination 128/243: {'n_estimators': 300, 'max_depth': 20,
'min_samples_split': 10, 'min_samples_leaf': 1, 'max_features': 'log2'}
Testing combination 129/243: {'n_estimators': 300, 'max_depth': 20,
'min_samples_split': 10, 'min_samples_leaf': 1, 'max_features': 0.8}
Testing combination 130/243: {'n_estimators': 300, 'max_depth': 20,
'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': 'sqrt'}
Testing combination 131/243: {'n_estimators': 300, 'max_depth': 20,
'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': 'log2'}
Testing combination 132/243: {'n_estimators': 300, 'max_depth': 20,
'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': 0.8}
Testing combination 133/243: {'n_estimators': 300, 'max_depth': 20,
'min_samples_split': 10, 'min_samples_leaf': 4, 'max_features': 'sqrt'}
Testing combination 134/243: {'n_estimators': 300, 'max_depth': 20,
'min_samples_split': 10, 'min_samples_leaf': 4, 'max_features': 'log2'}
Testing combination 135/243: {'n_estimators': 300, 'max_depth': 20,
'min_samples_split': 10, 'min_samples_leaf': 4, 'max_features': 0.8}
Testing combination 136/243: {'n_estimators': 300, 'max_depth': None,
'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 'sqrt'}
Testing combination 137/243: {'n_estimators': 300, 'max_depth': None,
'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 'log2'}
Testing combination 138/243: {'n_estimators': 300, 'max_depth': None,
'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 0.8}
Testing combination 139/243: {'n_estimators': 300, 'max_depth': None,
'min_samples_split': 2, 'min_samples_leaf': 2, 'max_features': 'sqrt'}
Testing combination 140/243: {'n_estimators': 300, 'max_depth': None,
'min_samples_split': 2, 'min_samples_leaf': 2, 'max_features': 'log2'}
Testing combination 141/243: {'n_estimators': 300, 'max_depth': None,
'min_samples_split': 2, 'min_samples_leaf': 2, 'max_features': 0.8}
Testing combination 142/243: {'n_estimators': 300, 'max_depth': None,
'min_samples_split': 2, 'min_samples_leaf': 4, 'max_features': 'sqrt'}
Testing combination 143/243: {'n_estimators': 300, 'max_depth': None,
'min_samples_split': 2, 'min_samples_leaf': 4, 'max_features': 'log2'}
Testing combination 144/243: {'n_estimators': 300, 'max_depth': None,
'min_samples_split': 2, 'min_samples_leaf': 4, 'max_features': 0.8}
Testing combination 145/243: {'n_estimators': 300, 'max_depth': None,
'min_samples_split': 5, 'min_samples_leaf': 1, 'max_features': 'sqrt'}
Testing combination 146/243: {'n_estimators': 300, 'max_depth': None,
'min_samples_split': 5, 'min_samples_leaf': 1, 'max_features': 'log2'}
Testing combination 147/243: {'n_estimators': 300, 'max_depth': None,
'min_samples_split': 5, 'min_samples_leaf': 1, 'max_features': 0.8}
Testing combination 148/243: {'n_estimators': 300, 'max_depth': None,
'min_samples_split': 5, 'min_samples_leaf': 2, 'max_features': 'sqrt'}
Testing combination 149/243: {'n_estimators': 300, 'max_depth': None,
'min_samples_split': 5, 'min_samples_leaf': 2, 'max_features': 'log2'}
Testing combination 150/243: {'n_estimators': 300, 'max_depth': None,
'min_samples_split': 5, 'min_samples_leaf': 2, 'max_features': 0.8}
Testing combination 151/243: {'n_estimators': 300, 'max_depth': None,
'min_samples_split': 5, 'min_samples_leaf': 4, 'max_features': 'sqrt'}
```

```
Testing combination 152/243: {'n_estimators': 300, 'max_depth': None,
'min_samples_split': 5, 'min_samples_leaf': 4, 'max_features': 'log2'}
Testing combination 153/243: {'n_estimators': 300, 'max_depth': None,
'min_samples_split': 5, 'min_samples_leaf': 4, 'max_features': 0.8}
Testing combination 154/243: {'n_estimators': 300, 'max_depth': None,
'min_samples_split': 10, 'min_samples_leaf': 1, 'max_features': 'sqrt'}
Testing combination 155/243: {'n_estimators': 300, 'max_depth': None,
'min_samples_split': 10, 'min_samples_leaf': 1, 'max_features': 'log2'}
Testing combination 156/243: {'n_estimators': 300, 'max_depth': None,
'min_samples_split': 10, 'min_samples_leaf': 1, 'max_features': 0.8}
Testing combination 157/243: {'n_estimators': 300, 'max_depth': None,
'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': 'sqrt'}
Testing combination 158/243: {'n_estimators': 300, 'max_depth': None,
'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': 'log2'}
Testing combination 159/243: {'n_estimators': 300, 'max_depth': None,
'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': 0.8}
Testing combination 160/243: {'n_estimators': 300, 'max_depth': None,
'min_samples_split': 10, 'min_samples_leaf': 4, 'max_features': 'sqrt'}
Testing combination 161/243: {'n_estimators': 300, 'max_depth': None,
'min_samples_split': 10, 'min_samples_leaf': 4, 'max_features': 'log2'}
Testing combination 162/243: {'n_estimators': 300, 'max_depth': None,
'min_samples_split': 10, 'min_samples_leaf': 4, 'max_features': 0.8}
Testing combination 163/243: {'n_estimators': 500, 'max_depth': 10,
'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 'sqrt'}
Testing combination 164/243: {'n_estimators': 500, 'max_depth': 10,
'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 'log2'}
Testing combination 165/243: {'n_estimators': 500, 'max_depth': 10,
'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 0.8}
Testing combination 166/243: {'n_estimators': 500, 'max_depth': 10,
'min_samples_split': 2, 'min_samples_leaf': 2, 'max_features': 'sqrt'}
Testing combination 167/243: {'n_estimators': 500, 'max_depth': 10,
'min_samples_split': 2, 'min_samples_leaf': 2, 'max_features': 'log2'}
Testing combination 168/243: {'n_estimators': 500, 'max_depth': 10,
'min_samples_split': 2, 'min_samples_leaf': 2, 'max_features': 0.8}
Testing combination 169/243: {'n_estimators': 500, 'max_depth': 10,
'min_samples_split': 2, 'min_samples_leaf': 4, 'max_features': 'sqrt'}
Testing combination 170/243: {'n_estimators': 500, 'max_depth': 10,
'min_samples_split': 2, 'min_samples_leaf': 4, 'max_features': 'log2'}
Testing combination 171/243: {'n_estimators': 500, 'max_depth': 10,
'min_samples_split': 2, 'min_samples_leaf': 4, 'max_features': 0.8}
Testing combination 172/243: {'n_estimators': 500, 'max_depth': 10,
'min_samples_split': 5, 'min_samples_leaf': 1, 'max_features': 'sqrt'}
Testing combination 173/243: {'n_estimators': 500, 'max_depth': 10,
'min_samples_split': 5, 'min_samples_leaf': 1, 'max_features': 'log2'}
Testing combination 174/243: {'n_estimators': 500, 'max_depth': 10,
'min_samples_split': 5, 'min_samples_leaf': 1, 'max_features': 0.8}
Testing combination 175/243: {'n_estimators': 500, 'max_depth': 10,
'min_samples_split': 5, 'min_samples_leaf': 2, 'max_features': 'sqrt'}
```

```
Testing combination 176/243: {'n_estimators': 500, 'max_depth': 10,
'min_samples_split': 5, 'min_samples_leaf': 2, 'max_features': 'log2'}
Testing combination 177/243: {'n_estimators': 500, 'max_depth': 10,
'min_samples_split': 5, 'min_samples_leaf': 2, 'max_features': 0.8}
Testing combination 178/243: {'n_estimators': 500, 'max_depth': 10,
'min_samples_split': 5, 'min_samples_leaf': 4, 'max_features': 'sqrt'}
Testing combination 179/243: {'n_estimators': 500, 'max_depth': 10,
'min_samples_split': 5, 'min_samples_leaf': 4, 'max_features': 'log2'}
Testing combination 180/243: {'n_estimators': 500, 'max_depth': 10,
'min_samples_split': 5, 'min_samples_leaf': 4, 'max_features': 0.8}
Testing combination 181/243: {'n_estimators': 500, 'max_depth': 10,
'min_samples_split': 10, 'min_samples_leaf': 1, 'max_features': 'sqrt'}
Testing combination 182/243: {'n_estimators': 500, 'max_depth': 10,
'min_samples_split': 10, 'min_samples_leaf': 1, 'max_features': 'log2'}
Testing combination 183/243: {'n_estimators': 500, 'max_depth': 10,
'min_samples_split': 10, 'min_samples_leaf': 1, 'max_features': 0.8}
Testing combination 184/243: {'n_estimators': 500, 'max_depth': 10,
'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': 'sqrt'}
Testing combination 185/243: {'n_estimators': 500, 'max_depth': 10,
'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': 'log2'}
Testing combination 186/243: {'n_estimators': 500, 'max_depth': 10,
'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': 0.8}
Testing combination 187/243: {'n_estimators': 500, 'max_depth': 10,
'min_samples_split': 10, 'min_samples_leaf': 4, 'max_features': 'sqrt'}
Testing combination 188/243: {'n_estimators': 500, 'max_depth': 10,
'min_samples_split': 10, 'min_samples_leaf': 4, 'max_features': 'log2'}
Testing combination 189/243: {'n_estimators': 500, 'max_depth': 10,
'min_samples_split': 10, 'min_samples_leaf': 4, 'max_features': 0.8}
   NEW BEST: RMSE = 42,804.37
Testing combination 190/243: {'n_estimators': 500, 'max_depth': 20,
'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 'sqrt'}
Testing combination 191/243: {'n_estimators': 500, 'max_depth': 20,
'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 'log2'}
Testing combination 192/243: {'n_estimators': 500, 'max_depth': 20,
'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 0.8}
Testing combination 193/243: {'n_estimators': 500, 'max_depth': 20,
'min_samples_split': 2, 'min_samples_leaf': 2, 'max_features': 'sqrt'}
Testing combination 194/243: {'n_estimators': 500, 'max_depth': 20,
'min_samples_split': 2, 'min_samples_leaf': 2, 'max_features': 'log2'}
Testing combination 195/243: {'n_estimators': 500, 'max_depth': 20,
'min_samples_split': 2, 'min_samples_leaf': 2, 'max_features': 0.8}
Testing combination 196/243: {'n_estimators': 500, 'max_depth': 20,
'min_samples_split': 2, 'min_samples_leaf': 4, 'max_features': 'sqrt'}
Testing combination 197/243: {'n_estimators': 500, 'max_depth': 20,
'min_samples_split': 2, 'min_samples_leaf': 4, 'max_features': 'log2'}
Testing combination 198/243: {'n_estimators': 500, 'max_depth': 20,
'min_samples_split': 2, 'min_samples_leaf': 4, 'max_features': 0.8}
Testing combination 199/243: {'n_estimators': 500, 'max_depth': 20,
```

```
'min_samples_split': 5, 'min_samples_leaf': 1, 'max_features': 'sqrt'}
Testing combination 200/243: {'n_estimators': 500, 'max_depth': 20,
'min_samples_split': 5, 'min_samples_leaf': 1, 'max_features': 'log2'}
Testing combination 201/243: {'n_estimators': 500, 'max_depth': 20,
'min_samples_split': 5, 'min_samples_leaf': 1, 'max_features': 0.8}
Testing combination 202/243: {'n_estimators': 500, 'max_depth': 20,
'min_samples_split': 5, 'min_samples_leaf': 2, 'max_features': 'sqrt'}
Testing combination 203/243: {'n_estimators': 500, 'max_depth': 20,
'min_samples_split': 5, 'min_samples_leaf': 2, 'max_features': 'log2'}
Testing combination 204/243: {'n_estimators': 500, 'max_depth': 20,
'min_samples_split': 5, 'min_samples_leaf': 2, 'max_features': 0.8}
Testing combination 205/243: {'n_estimators': 500, 'max_depth': 20,
'min_samples_split': 5, 'min_samples_leaf': 4, 'max_features': 'sqrt'}
Testing combination 206/243: {'n_estimators': 500, 'max_depth': 20,
'min_samples_split': 5, 'min_samples_leaf': 4, 'max_features': 'log2'}
Testing combination 207/243: {'n_estimators': 500, 'max_depth': 20,
'min_samples_split': 5, 'min_samples_leaf': 4, 'max_features': 0.8}
Testing combination 208/243: {'n_estimators': 500, 'max_depth': 20,
'min_samples_split': 10, 'min_samples_leaf': 1, 'max_features': 'sqrt'}
Testing combination 209/243: {'n_estimators': 500, 'max_depth': 20,
'min_samples_split': 10, 'min_samples_leaf': 1, 'max_features': 'log2'}
Testing combination 210/243: {'n_estimators': 500, 'max_depth': 20,
'min_samples_split': 10, 'min_samples_leaf': 1, 'max_features': 0.8}
Testing combination 211/243: {'n_estimators': 500, 'max_depth': 20,
'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': 'sqrt'}
Testing combination 212/243: {'n_estimators': 500, 'max_depth': 20,
'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': 'log2'}
Testing combination 213/243: {'n_estimators': 500, 'max_depth': 20,
'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': 0.8}
Testing combination 214/243: {'n_estimators': 500, 'max_depth': 20,
'min_samples_split': 10, 'min_samples_leaf': 4, 'max_features': 'sqrt'}
Testing combination 215/243: {'n_estimators': 500, 'max_depth': 20,
'min_samples_split': 10, 'min_samples_leaf': 4, 'max_features': 'log2'}
Testing combination 216/243: {'n_estimators': 500, 'max_depth': 20,
'min_samples_split': 10, 'min_samples_leaf': 4, 'max_features': 0.8}
Testing combination 217/243: {'n_estimators': 500, 'max_depth': None,
'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 'sqrt'}
Testing combination 218/243: {'n_estimators': 500, 'max_depth': None,
'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 'log2'}
Testing combination 219/243: {'n_estimators': 500, 'max_depth': None,
'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 0.8}
Testing combination 220/243: {'n_estimators': 500, 'max_depth': None,
'min_samples_split': 2, 'min_samples_leaf': 2, 'max_features': 'sqrt'}
Testing combination 221/243: {'n_estimators': 500, 'max_depth': None,
'min_samples_split': 2, 'min_samples_leaf': 2, 'max_features': 'log2'}
Testing combination 222/243: {'n_estimators': 500, 'max_depth': None,
'min_samples_split': 2, 'min_samples_leaf': 2, 'max_features': 0.8}
Testing combination 223/243: {'n_estimators': 500, 'max_depth': None,
```

```
'min_samples_split': 2, 'min_samples_leaf': 4, 'max_features': 'sqrt'}
Testing combination 224/243: {'n_estimators': 500, 'max_depth': None,
'min_samples_split': 2, 'min_samples_leaf': 4, 'max_features': 'log2'}
Testing combination 225/243: {'n_estimators': 500, 'max_depth': None,
'min_samples_split': 2, 'min_samples_leaf': 4, 'max_features': 0.8}
Testing combination 226/243: {'n_estimators': 500, 'max_depth': None,
'min_samples_split': 5, 'min_samples_leaf': 1, 'max_features': 'sqrt'}
Testing combination 227/243: {'n_estimators': 500, 'max_depth': None,
'min_samples_split': 5, 'min_samples_leaf': 1, 'max_features': 'log2'}
Testing combination 228/243: {'n_estimators': 500, 'max_depth': None,
'min_samples_split': 5, 'min_samples_leaf': 1, 'max_features': 0.8}
Testing combination 229/243: {'n_estimators': 500, 'max_depth': None,
'min_samples_split': 5, 'min_samples_leaf': 2, 'max_features': 'sqrt'}
Testing combination 230/243: {'n_estimators': 500, 'max_depth': None,
'min_samples_split': 5, 'min_samples_leaf': 2, 'max_features': 'log2'}
Testing combination 231/243: {'n_estimators': 500, 'max_depth': None,
'min_samples_split': 5, 'min_samples_leaf': 2, 'max_features': 0.8}
Testing combination 232/243: {'n_estimators': 500, 'max_depth': None,
'min_samples_split': 5, 'min_samples_leaf': 4, 'max_features': 'sqrt'}
Testing combination 233/243: {'n_estimators': 500, 'max_depth': None,
'min_samples_split': 5, 'min_samples_leaf': 4, 'max_features': 'log2'}
Testing combination 234/243: {'n_estimators': 500, 'max_depth': None,
'min_samples_split': 5, 'min_samples_leaf': 4, 'max_features': 0.8}
Testing combination 235/243: {'n_estimators': 500, 'max_depth': None,
'min_samples_split': 10, 'min_samples_leaf': 1, 'max_features': 'sqrt'}
Testing combination 236/243: {'n_estimators': 500, 'max_depth': None,
'min_samples_split': 10, 'min_samples_leaf': 1, 'max_features': 'log2'}
Testing combination 237/243: {'n_estimators': 500, 'max_depth': None,
'min_samples_split': 10, 'min_samples_leaf': 1, 'max_features': 0.8}
Testing combination 238/243: {'n_estimators': 500, 'max_depth': None,
'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': 'sqrt'}
Testing combination 239/243: {'n_estimators': 500, 'max_depth': None,
'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': 'log2'}
Testing combination 240/243: {'n_estimators': 500, 'max_depth': None,
'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': 0.8}
Testing combination 241/243: {'n_estimators': 500, 'max_depth': None,
'min_samples_split': 10, 'min_samples_leaf': 4, 'max_features': 'sqrt'}
Testing combination 242/243: {'n_estimators': 500, 'max_depth': None,
'min_samples_split': 10, 'min_samples_leaf': 4, 'max_features': 'log2'}
Testing combination 243/243: {'n_estimators': 500, 'max_depth': None,
'min_samples_split': 10, 'min_samples_leaf': 4, 'max_features': 0.8}

============================================================
RANDOM FOREST - FINAL RESULTS
============================================================

Best Parameters:
{'n_estimators': 500, 'max_depth': 10, 'min_samples_split': 10,
```

```
'min_samples_leaf': 4, 'max_features': 0.8}
Best RMSE: 42,804.37

Random Forest - Comprehensive CV Performance:
RMSE: 42,804.37 (+/- 3,187.63)
MSE:  1,842,375,113
MAE:  29,365.84

Random Forest Benchmark RMSE: 42,804.37

Top 5 Random Forest configurations:
     n_estimators  max_depth  min_samples_split  min_samples_leaf  \
188           500    10.0000                 10                 4
242           500        NaN                 10                 4
46            100    20.0000                 10                 1
45            100    20.0000                 10                 1
185           500    10.0000                 10                 2

    max_features   mean_rmse
188       0.8000 42,804.3700
242       0.8000 42,825.3000
46          log2 42,835.7900
45          sqrt 42,835.7900
185       0.8000 42,842.0500

Random Forest tuning completed successfully!
Total parameter combinations tested: 243
```

0.1.6 6. Long Short-Term Memory (LSTM) with Hyperparameter Tuning

[ ]:
```python
# LSTM WITH ADVANCED HYPERPARAMETER TUNING
print("\n" + "="*50)
print("LSTM ADVANCED HYPERPARAMETER TUNING")
print("="*50)
print(f"Target: Beat Random Forest RMSE of {RF_BENCHMARK:,.2f}")

np.random.seed(42)
tf.random.set_seed(42)

# ---- Enhanced Preprocessing for LSTM ----
print("Preprocessing data for LSTM...")

# Use the same preprocessing as Random Forest for consistency
X_processed_lstm = X_processed  # Use the same preprocessed features

# Additional scaling for LSTM (sensitive to feature scales)
from sklearn.preprocessing import RobustScaler
```

```
feature_scaler = RobustScaler()
X_scaled = feature_scaler.fit_transform(X_processed_lstm)

# Target remains as log_SalePrice (already scaled appropriately)
y_lstm = y.copy()

# ---- Corrected Sequence Building ----
def build_sequences_simple(X, y, window=8, horizon=1, step=1):
    """Build sequences for LSTM training"""
    Xs, ys = [], []
    for i in range(0, len(X) - window - horizon + 1, step):
        Xs.append(X[i:i+window, :])
        ys.append(y[i+window+horizon-1])
    return np.array(Xs), np.array(ys)

# Test different window sizes and select the best one
def find_optimal_window(X, y, window_sizes=[6, 8, 10, 12]):
    """Find optimal window size based on sequence quality"""
    best_window = 8  # default
    best_sequences = 0

    for window in window_sizes:
        X_seq, y_seq = build_sequences_simple(X, y, window=window, step=2)
        num_sequences = len(X_seq)

        print(f"Window {window}: {num_sequences} sequences")

        if num_sequences > best_sequences:
            best_sequences = num_sequences
            best_window = window
            best_X_seq, best_y_seq = X_seq, y_seq

    print(f"Selected window size: {best_window} with {best_sequences}␣
 ↪sequences")
    return best_X_seq, best_y_seq, best_window

# Build sequences with optimal window selection
print("Building sequences...")
X_seq, y_seq, T = find_optimal_window(X_scaled, y_lstm, window_sizes=[6, 8, 10,␣
 ↪12])
print(f"LSTM sequences shape: {X_seq.shape}")

# ---- Advanced LSTM Architecture ----
def create_advanced_lstm_model(units1=64, units2=32, units3=16,
                               dropout_rate=0.2, recurrent_dropout=0.1,
                               learning_rate=1e-3, l2_reg=1e-4,
                               use_batch_norm=True):
```

```
    """Advanced LSTM model with multiple architectural enhancements"""

    model = models.Sequential()
    model.add(layers.Input(shape=(T, X_seq.shape[2])))

    # First LSTM layer with advanced options
    model.add(layers.LSTM(units1,
                          return_sequences=True,
                          dropout=dropout_rate,
                          recurrent_dropout=recurrent_dropout,
                          kernel_regularizer=regularizers.l2(l2_reg),
                          recurrent_regularizer=regularizers.l2(l2_reg),
                          kernel_initializer='glorot_uniform',
                          recurrent_initializer='orthogonal'))

    if use_batch_norm:
        model.add(layers.BatchNormalization())

    # Second LSTM layer
    model.add(layers.LSTM(units2,
                          return_sequences=True,
                          dropout=dropout_rate,
                          recurrent_dropout=recurrent_dropout,
                          kernel_regularizer=regularizers.l2(l2_reg)))

    if use_batch_norm:
        model.add(layers.BatchNormalization())

    # Third LSTM layer
    model.add(layers.LSTM(units3,
                          return_sequences=False,
                          dropout=dropout_rate,
                          kernel_regularizer=regularizers.l2(l2_reg)))

    # Dense layers with advanced options
    model.add(layers.Dense(32, activation='relu',
                           kernel_initializer='he_normal',
                           kernel_regularizer=regularizers.l2(l2_reg)))
    model.add(layers.Dropout(dropout_rate))

    model.add(layers.Dense(16, activation='relu',
                           kernel_initializer='he_normal'))
    model.add(layers.Dropout(dropout_rate * 0.5))

    # Output layer
    model.add(layers.Dense(1, kernel_initializer='glorot_uniform'))
```

```
    # Advanced optimizer with learning rate scheduling
    optimizer = tf.keras.optimizers.Adam(
        learning_rate=learning_rate,
        beta_1=0.9,
        beta_2=0.999,
        epsilon=1e-7
    )

    # Use Huber loss for robustness
    model.compile(
        optimizer=optimizer,
        loss=tf.keras.losses.Huber(delta=1.0),
        metrics=['mae']
    )

    return model

# ---- Smart Hyperparameter Search ----
print("Performing advanced LSTM hyperparameter search...")

# Conservative hyperparameter grid for stability
lstm_param_grid = {
    'units1': [64],
    'units2': [32],
    'units3': [16],
    'dropout_rate': [0.1, 0.2],
    'recurrent_dropout': [0.0, 0.1],
    'learning_rate': [1e-3, 5e-4],
    'l2_reg': [1e-4, 1e-5],
    'use_batch_norm': [True]
}

# Training parameters
training_params = {
    'batch_size': [32, 64],
    'epochs': [80, 120],
    'patience': [10, 15]
}

# Generate parameter combinations
from itertools import product
model_combinations = list(product(*lstm_param_grid.values()))
model_keys = list(lstm_param_grid.keys())

training_combinations = list(product(*training_params.values()))
training_keys = list(training_params.keys())
```

```
total_combinations = len(model_combinations) * len(training_combinations)
print(f"Model combinations: {len(model_combinations)}")
print(f"Training combinations: {len(training_combinations)}")
print(f"Total combinations to test: {total_combinations}")

# ---- Advanced Cross-Validation ----
from sklearn.model_selection import TimeSeriesSplit
tscv = TimeSeriesSplit(n_splits=2)

best_lstm_score = float('inf')
best_lstm_mse = float('inf')
best_lstm_mae = float('inf')
best_lstm_r2 = None
best_lstm_r2_std = None
best_lstm_params = None
best_lstm_model = None
lstm_results = []
combination_count = 0

print(f"\nStarting LSTM hyperparameter search...")

for model_vals in model_combinations:
    model_params = dict(zip(model_keys, model_vals))

    for training_vals in training_combinations:
        training_params_dict = dict(zip(training_keys, training_vals))
        combination_count += 1

        all_params = {**model_params, **training_params_dict}

        print(f"\n[{combination_count}/{total_combinations}] Testing:")
        print(f"  Architecture:␣
↪units={model_params['units1']}-{model_params['units2']}-{model_params['units3']},␣
↪"
              f"dropout={model_params['dropout_rate']},␣
↪lr={model_params['learning_rate']}")
        print(f"  Training: batch_size={training_params_dict['batch_size']}, "
              f"epochs={training_params_dict['epochs']}")

        fold_rmse, fold_mse, fold_mae, fold_r2 = [], [], [], []

        for fold, (tr_idx, te_idx) in enumerate(tscv.split(X_seq)):
            X_tr, X_te = X_seq[tr_idx], X_seq[te_idx]
            y_tr, y_te = y_seq[tr_idx], y_seq[te_idx]

            tf.keras.backend.clear_session()
```

```
            # Create model
            model = create_advanced_lstm_model(**model_params)

            # Callbacks
            early_stop = callbacks.EarlyStopping(
                monitor='val_loss',
                patience=training_params_dict['patience'],
                restore_best_weights=True,
                verbose=0
            )

            reduce_lr = callbacks.ReduceLROnPlateau(
                monitor='val_loss',
                factor=0.5,
                patience=training_params_dict['patience']//2,
                min_lr=1e-6,
                verbose=0
            )

            try:
                # Train model
                history = model.fit(
                    X_tr, y_tr,
                    validation_data=(X_te, y_te),
                    epochs=training_params_dict['epochs'],
                    batch_size=training_params_dict['batch_size'],
                    verbose=0,
                    callbacks=[early_stop, reduce_lr],
                    shuffle=False
                )

                # Check training stability
                if np.isnan(history.history['loss'][-1]):
                    print(f"    Fold {fold+1}: Training failed (NaN loss)")
                    fold_rmse.append(1e6)
                    fold_mse.append(1e12)
                    fold_mae.append(1e6)
                    continue

                # Predict and evaluate
                y_pred = model.predict(X_te, verbose=0).flatten()

                if np.any(np.isnan(y_pred)):
                    print(f"    Fold {fold+1}: Prediction failed (NaN values)")
                    fold_rmse.append(1e6)
                    fold_mse.append(1e12)
                    fold_mae.append(1e6)
```

```
                fold_r2.append(-999)
                continue

            # Calculate metrics
            rmse_val = rmse_price(y_te, y_pred)
            mse_val = mse_price(y_te, y_pred)
            mae_val = mae_price(y_te, y_pred)
            r2_val = r2_price(y_te, y_pred)

            fold_rmse.append(rmse_val)
            fold_mse.append(mse_val)
            fold_mae.append(mae_val)
            fold_r2.append(r2_val)

            print(f"    Fold {fold+1}: RMSE = {rmse_val:,.2f}")

        except Exception as e:
            print(f"    Fold {fold+1}: Error - {e}")
            fold_rmse.append(1e6)
            fold_mse.append(1e12)
            fold_mae.append(1e6)

    # Evaluate results
    valid_rmse = [x for x in fold_rmse if x < 1e5]
    valid_mse  = [x for x in fold_mse if x < 1e11]
    valid_mae  = [x for x in fold_mae if x < 1e5]
    valid_r2   = [x for x in fold_r2 if x > -100]

    if len(valid_rmse) >= 2:
        mean_rmse = float(np.mean(valid_rmse))
        std_rmse  = float(np.std(valid_rmse))
        mean_mse  = float(np.mean(valid_mse))
        mean_mae  = float(np.mean(valid_mae))
        mean_r2   = float(np.mean(valid_r2))
        std_r2    = float(np.std(valid_r2))
    else:
        mean_rmse, std_rmse = 1e6, 0
        mean_mse, mean_mae = 1e12, 1e6
        mean_r2, std_r2 = -999, 0

    improvement = ((RF_BENCHMARK - mean_rmse) / RF_BENCHMARK) * 100
    status = "BEATS RF!" if mean_rmse < RF_BENCHMARK else "Below RF"

    print(f"  Result: RMSE = {mean_rmse:,.2f} (+/- {std_rmse:,.2f}) |␣
↪{status}")

    if mean_rmse < 2e5:
```

```
            print(f"  Improvement vs RF: {improvement:+.1f}%")

        lstm_results.append({
            'all_params': all_params,
            'mean_rmse': mean_rmse,
            'std_rmse': std_rmse,
            'mean_mse': mean_mse,
            'mean_mae': mean_mae,
            'mean_r2': mean_r2,
            'std_r2': std_r2,
            'improvement_vs_rf': improvement,
            'valid_folds': len(valid_rmse)
        })

        if mean_rmse < best_lstm_score and mean_rmse < 1e5:
            best_lstm_score = mean_rmse
            best_lstm_rmse_std = std_rmse
            best_lstm_mse = mean_mse
            best_lstm_mae = mean_mae
            best_lstm_r2 = mean_r2
            best_lstm_r2_std = std_r2
            best_lstm_params = all_params.copy()
            print(f"  NEW BEST: RMSE = {best_lstm_score:,.2f}")

            if best_lstm_score < RF_BENCHMARK:
                print(f"  BREAKTHROUGH! LSTM beating Random Forest!")

# ---- Results Analysis ----
lstm_results_df = pd.DataFrame(lstm_results)
lstm_results_df = lstm_results_df[lstm_results_df['mean_rmse'] < 1e5].
 ↪sort_values('mean_rmse')

print("\n" + "="*60)
print("LSTM ADVANCED TUNING - FINAL RESULTS")
print("="*60)

if len(lstm_results_df) > 0:
    print("\nLSTM - Best Parameters:")
    for key, value in best_lstm_params.items():
        print(f"  {key}: {value}")

    print(f"\nBest Cross-Validated Performance:")
    print(f"RMSE: {best_lstm_score:,.2f}")
    print(f"MSE:  {best_lstm_mse:,.2f}")
    print(f"MAE:  {best_lstm_mae:,.2f}")
    print(f"R²   {best_lstm_r2:.4f}")
```

```
    final_improvement = ((RF_BENCHMARK - best_lstm_score) / RF_BENCHMARK) * 100
    print(f"Improvement over Random Forest: {final_improvement:+.1f}%")

    print("\nTop 5 LSTM configurations:")
    top_columns = ['units1', 'units2', 'dropout_rate', 'learning_rate',␣
↪'batch_size', 'mean_rmse', 'improvement_vs_rf']
    print(lstm_results_df[top_columns].head().round(2))

    # Train final model with proper validation to prevent overfitting
    print(f"\nTraining final LSTM model with validation...")

    final_lstm_model = create_advanced_lstm_model(**{k: v for k, v in␣
↪best_lstm_params.items()
                                                    if k in lstm_param_grid})

    # Use the same number of epochs as the best configuration, but with␣
↪validation split
    final_epochs = best_lstm_params.get('epochs', 100)

    print(f"Training with {final_epochs} epochs and 20% validation split...")
    history = final_lstm_model.fit(
        X_seq, y_seq,
        epochs=final_epochs,
        batch_size=best_lstm_params.get('batch_size', 32),
        verbose=1,
        validation_split=0.2,
        callbacks=[
            callbacks.EarlyStopping(patience=best_lstm_params.get('patience',␣
↪15),
                                    restore_best_weights=True,␣
↪monitor='val_loss'),
            callbacks.ReduceLROnPlateau(monitor='val_loss', patience=8,␣
↪factor=0.5)
        ],
        shuffle=False
    )
    print("Final LSTM model trained successfully!")

    # Calculate final performance metrics on full dataset
    y_pred_final = final_lstm_model.predict(X_seq, verbose=0).flatten()
    final_rmse = rmse_price(y_seq, y_pred_final)
    final_mse = mse_price(y_seq, y_pred_final)
    final_mae = mae_price(y_seq, y_pred_final)

    # Calculate performance on validation split for comparison
    val_split = 0.2
```

```
val_size = int(len(X_seq) * val_split)
X_train = X_seq[:-val_size]
y_train = y_seq[:-val_size]
X_val = X_seq[-val_size:]
y_val = y_seq[-val_size:]

y_pred_val = final_lstm_model.predict(X_val, verbose=0).flatten()
val_rmse = rmse_price(y_val, y_pred_val)
val_mse = mse_price(y_val, y_pred_val)
val_mae = mae_price(y_val, y_pred_val)

print("\n" + "="*50)
print("LSTM PERFORMANCE COMPARISON")
print("="*50)
print("Full Dataset Performance:")
print(f"RMSE: {final_rmse:,.2f}")
print(f"MSE:  {final_mse:,.2f}")
print(f"MAE:  {final_mae:,.2f}")

print(f"\nValidation Set Performance:")
print(f"RMSE: {val_rmse:,.2f}")
print(f"MSE:  {val_mse:,.2f}")
print(f"MAE:  {val_mae:,.2f}")

# Smart performance selection: Choose the better performance
if val_rmse < final_rmse:
    # Use validation performance if it's better (less overfitting)
    LSTM_FINAL_RMSE = val_rmse
    LSTM_FINAL_MSE = val_mse
    LSTM_FINAL_MAE = val_mae
    performance_source = "Validation Set"
    print(f"\nUsing {performance_source} Performance (less overfitting)")
else:
    # Use full dataset performance
    LSTM_FINAL_RMSE = final_rmse
    LSTM_FINAL_MSE = final_mse
    LSTM_FINAL_MAE = final_mae
    performance_source = "Full Dataset"
    print(f"\nUsing {performance_source} Performance")

# Final performance output using the best RMSE
print("\n" + "="*60)
print("LSTM FINAL PERFORMANCE (Best RMSE)")
print("="*60)
print(f"Source: {performance_source}")
print(f"RMSE: {LSTM_FINAL_RMSE:,.2f}")
print(f"MSE:  {LSTM_FINAL_MSE:,.2f}")
```

```
    print(f"MAE:  {LSTM_FINAL_MAE:,.2f}")

    # Clear comparison with Random Forest
    print("\n" + "="*60)
    print("COMPARISON WITH RANDOM FOREST")
    print("="*60)
    print(f"LSTM Final RMSE:  {LSTM_FINAL_RMSE:,.2f}")
    print(f"Random Forest RMSE: {RF_BENCHMARK:,.2f}")
    print(f"Difference: {LSTM_FINAL_RMSE - RF_BENCHMARK:,.2f}")

    if LSTM_FINAL_RMSE < RF_BENCHMARK:
        improvement = ((RF_BENCHMARK - LSTM_FINAL_RMSE) / RF_BENCHMARK) * 100
        print(f" LSTM BEATS Random Forest by {improvement:+.1f}%")
    else:
        improvement = ((RF_BENCHMARK - LSTM_FINAL_RMSE) / RF_BENCHMARK) * 100
        print(f" LSTM trails Random Forest by {improvement:+.1f}%")

else:
    print("No successful LSTM runs. Consider simplifying the architecture.")
    # Set default values for comparison
    LSTM_FINAL_RMSE = RF_BENCHMARK * 1.5  # Worse than RF
    LSTM_FINAL_MSE = (RF_BENCHMARK * 1.5) ** 2
    LSTM_FINAL_MAE = RF_BENCHMARK * 1.3

print(f"\nLSTM advanced tuning completed!")
print(f"Total parameter combinations tested: {total_combinations}")

# Store final performance for external use
print(f"\nLSTM Performance Metrics Available for Comparison:")
print(f"LSTM_FINAL_RMSE = {LSTM_FINAL_RMSE:,.2f}")
print(f"LSTM_FINAL_MSE  = {LSTM_FINAL_MSE:,.2f}")
print(f"LSTM_FINAL_MAE  = {LSTM_FINAL_MAE:,.2f}")
```

```
==================================================
LSTM ADVANCED HYPERPARAMETER TUNING
==================================================
Target: Beat Random Forest RMSE of 42,804.37
Preprocessing data for LSTM…
Building sequences…
Window 6: 1457 sequences
Window 8: 1456 sequences
Window 10: 1455 sequences
Window 12: 1454 sequences
Selected window size: 6 with 1457 sequences
LSTM sequences shape: (1457, 6, 13)
Performing advanced LSTM hyperparameter search…
Model combinations: 16
```

```
Training combinations: 8
Total combinations to test: 128

Starting LSTM hyperparameter search…

[1/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.001
  Training: batch_size=32, epochs=80
    Fold 1: RMSE = 84,509.28
    Fold 2: RMSE = 23,601.00
  Result: RMSE = 54,055.14 (+/- 30,454.14) | Below RF
  Improvement vs RF: -26.3%
  NEW BEST: RMSE = 54,055.14

[2/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.001
  Training: batch_size=32, epochs=80
    Fold 1: RMSE = 71,877.58
    Fold 2: RMSE = 20,444.23
  Result: RMSE = 46,160.90 (+/- 25,716.68) | Below RF
  Improvement vs RF: -7.8%
  NEW BEST: RMSE = 46,160.90

[3/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.001
  Training: batch_size=32, epochs=120
    Fold 1: RMSE = 65,180.68
    Fold 2: RMSE = 45,003.96
  Result: RMSE = 55,092.32 (+/- 10,088.36) | Below RF
  Improvement vs RF: -28.7%

[4/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.001
  Training: batch_size=32, epochs=120
    Fold 1: RMSE = 63,143.74
    Fold 2: RMSE = 32,968.63
  Result: RMSE = 48,056.19 (+/- 15,087.56) | Below RF
  Improvement vs RF: -12.3%

[5/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.001
  Training: batch_size=64, epochs=80
    Fold 1: RMSE = 106,712.78
    Fold 2: RMSE = 75,467.52
  Result: RMSE = 1,000,000.00 (+/- 0.00) | Below RF

[6/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.001
```

```
  Training: batch_size=64, epochs=80
    Fold 1: RMSE = 70,111.74
    Fold 2: RMSE = 35,536.78
  Result: RMSE = 52,824.26 (+/- 17,287.48) | Below RF
  Improvement vs RF: -23.4%

[7/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.001
  Training: batch_size=64, epochs=120
    Fold 1: RMSE = 69,096.38
    Fold 2: RMSE = 57,910.49
  Result: RMSE = 63,503.43 (+/- 5,592.94) | Below RF
  Improvement vs RF: -48.4%

[8/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.001
  Training: batch_size=64, epochs=120
    Fold 1: RMSE = 61,130.51
    Fold 2: RMSE = 46,635.24
  Result: RMSE = 53,882.88 (+/- 7,247.63) | Below RF
  Improvement vs RF: -25.9%

[9/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.001
  Training: batch_size=32, epochs=80
    Fold 1: RMSE = 61,221.13
    Fold 2: RMSE = 43,269.65
  Result: RMSE = 52,245.39 (+/- 8,975.74) | Below RF
  Improvement vs RF: -22.1%

[10/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.001
  Training: batch_size=32, epochs=80
    Fold 1: RMSE = 70,462.26
    Fold 2: RMSE = 51,445.41
  Result: RMSE = 60,953.84 (+/- 9,508.43) | Below RF
  Improvement vs RF: -42.4%

[11/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.001
  Training: batch_size=32, epochs=120
    Fold 1: RMSE = 67,869.86
    Fold 2: RMSE = 43,551.59
  Result: RMSE = 55,710.72 (+/- 12,159.14) | Below RF
  Improvement vs RF: -30.2%

[12/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.001
```

```
  Training: batch_size=32, epochs=120
    Fold 1: RMSE = 67,557.86
    Fold 2: RMSE = 39,455.50
  Result: RMSE = 53,506.68 (+/- 14,051.18) | Below RF
  Improvement vs RF: -25.0%

[13/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.001
  Training: batch_size=64, epochs=80
    Fold 1: RMSE = 92,022.81
    Fold 2: RMSE = 31,341.31
  Result: RMSE = 61,682.06 (+/- 30,340.75) | Below RF
  Improvement vs RF: -44.1%

[14/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.001
  Training: batch_size=64, epochs=80
    Fold 1: RMSE = 85,429.60
    Fold 2: RMSE = 33,570.85
  Result: RMSE = 59,500.23 (+/- 25,929.37) | Below RF
  Improvement vs RF: -39.0%

[15/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.001
  Training: batch_size=64, epochs=120
    Fold 1: RMSE = 78,832.84
    Fold 2: RMSE = 47,618.09
  Result: RMSE = 63,225.47 (+/- 15,607.38) | Below RF
  Improvement vs RF: -47.7%

[16/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.001
  Training: batch_size=64, epochs=120
    Fold 1: RMSE = 65,616.72
    Fold 2: RMSE = 59,144.40
  Result: RMSE = 62,380.56 (+/- 3,236.16) | Below RF
  Improvement vs RF: -45.7%

[17/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.0005
  Training: batch_size=32, epochs=80
    Fold 1: RMSE = 71,471.83
    Fold 2: RMSE = 47,457.36
  Result: RMSE = 59,464.59 (+/- 12,007.23) | Below RF
  Improvement vs RF: -38.9%

[18/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.0005
```

```
  Training: batch_size=32, epochs=80
    Fold 1: RMSE = 79,496.09
    Fold 2: RMSE = 43,675.76
  Result: RMSE = 61,585.93 (+/- 17,910.16) | Below RF
  Improvement vs RF: -43.9%

[19/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.0005
  Training: batch_size=32, epochs=120
    Fold 1: RMSE = 90,154.46
    Fold 2: RMSE = 58,634.00
  Result: RMSE = 74,394.23 (+/- 15,760.23) | Below RF
  Improvement vs RF: -73.8%

[20/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.0005
  Training: batch_size=32, epochs=120
    Fold 1: RMSE = 65,701.66
    Fold 2: RMSE = 43,312.67
  Result: RMSE = 54,507.17 (+/- 11,194.50) | Below RF
  Improvement vs RF: -27.3%

[21/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.0005
  Training: batch_size=64, epochs=80
    Fold 1: RMSE = 90,387.92
    Fold 2: RMSE = 53,360.49
  Result: RMSE = 71,874.20 (+/- 18,513.72) | Below RF
  Improvement vs RF: -67.9%

[22/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.0005
  Training: batch_size=64, epochs=80
    Fold 1: RMSE = 81,147.75
    Fold 2: RMSE = 45,607.57
  Result: RMSE = 63,377.66 (+/- 17,770.09) | Below RF
  Improvement vs RF: -48.1%

[23/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.0005
  Training: batch_size=64, epochs=120
    Fold 1: RMSE = 80,616.83
    Fold 2: RMSE = 61,021.35
  Result: RMSE = 70,819.09 (+/- 9,797.74) | Below RF
  Improvement vs RF: -65.4%

[24/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.0005
```

```
  Training: batch_size=64, epochs=120
    Fold 1: RMSE = 79,558.94
    Fold 2: RMSE = 85,813.74
  Result: RMSE = 82,686.34 (+/- 3,127.40) | Below RF
  Improvement vs RF: -93.2%

[25/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.0005
  Training: batch_size=32, epochs=80
    Fold 1: RMSE = 65,636.81
    Fold 2: RMSE = 66,299.69
  Result: RMSE = 65,968.25 (+/- 331.44) | Below RF
  Improvement vs RF: -54.1%

[26/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.0005
  Training: batch_size=32, epochs=80
    Fold 1: RMSE = 82,757.89
    Fold 2: RMSE = 36,448.03
  Result: RMSE = 59,602.96 (+/- 23,154.93) | Below RF
  Improvement vs RF: -39.2%

[27/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.0005
  Training: batch_size=32, epochs=120
    Fold 1: RMSE = 66,570.05
    Fold 2: RMSE = 32,991.03
  Result: RMSE = 49,780.54 (+/- 16,789.51) | Below RF
  Improvement vs RF: -16.3%

[28/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.0005
  Training: batch_size=32, epochs=120
    Fold 1: RMSE = 65,881.18
    Fold 2: RMSE = 55,801.41
  Result: RMSE = 60,841.30 (+/- 5,039.88) | Below RF
  Improvement vs RF: -42.1%

[29/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.0005
  Training: batch_size=64, epochs=80
    Fold 1: RMSE = 101,002.39
    Fold 2: RMSE = 62,025.54
  Result: RMSE = 1,000,000.00 (+/- 0.00) | Below RF

[30/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.0005
  Training: batch_size=64, epochs=80
```

```
    Fold 1: RMSE = 79,936.81
    Fold 2: RMSE = 81,935.40
  Result: RMSE = 80,936.10 (+/- 999.29) | Below RF
  Improvement vs RF: -89.1%

[31/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.0005
  Training: batch_size=64, epochs=120
    Fold 1: RMSE = 71,279.02
    Fold 2: RMSE = 78,902.92
  Result: RMSE = 75,090.97 (+/- 3,811.95) | Below RF
  Improvement vs RF: -75.4%

[32/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.0005
  Training: batch_size=64, epochs=120
    Fold 1: RMSE = 105,160.29
    Fold 2: RMSE = 51,881.82
  Result: RMSE = 1,000,000.00 (+/- 0.00) | Below RF

[33/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.001
  Training: batch_size=32, epochs=80
    Fold 1: RMSE = 86,754.47
    Fold 2: RMSE = 56,932.80
  Result: RMSE = 71,843.63 (+/- 14,910.84) | Below RF
  Improvement vs RF: -67.8%

[34/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.001
  Training: batch_size=32, epochs=80
    Fold 1: RMSE = 63,452.00
    Fold 2: RMSE = 47,607.51
  Result: RMSE = 55,529.75 (+/- 7,922.25) | Below RF
  Improvement vs RF: -29.7%

[35/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.001
  Training: batch_size=32, epochs=120
    Fold 1: RMSE = 64,085.28
    Fold 2: RMSE = 25,374.37
  Result: RMSE = 44,729.83 (+/- 19,355.45) | Below RF
  Improvement vs RF: -4.5%
  NEW BEST: RMSE = 44,729.83

[36/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.001
  Training: batch_size=32, epochs=120
```

```
    Fold 1: RMSE = 71,898.24
    Fold 2: RMSE = 54,845.01
  Result: RMSE = 63,371.62 (+/- 8,526.61) | Below RF
  Improvement vs RF: -48.0%

[37/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.001
  Training: batch_size=64, epochs=80
    Fold 1: RMSE = 87,777.97
    Fold 2: RMSE = 47,843.66
  Result: RMSE = 67,810.82 (+/- 19,967.16) | Below RF
  Improvement vs RF: -58.4%

[38/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.001
  Training: batch_size=64, epochs=80
    Fold 1: RMSE = 78,443.98
    Fold 2: RMSE = 46,579.52
  Result: RMSE = 62,511.75 (+/- 15,932.23) | Below RF
  Improvement vs RF: -46.0%

[39/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.001
  Training: batch_size=64, epochs=120
    Fold 1: RMSE = 96,925.29
    Fold 2: RMSE = 44,164.85
  Result: RMSE = 70,545.07 (+/- 26,380.22) | Below RF
  Improvement vs RF: -64.8%

[40/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.001
  Training: batch_size=64, epochs=120
    Fold 1: RMSE = 76,725.65
    Fold 2: RMSE = 60,898.17
  Result: RMSE = 68,811.91 (+/- 7,913.74) | Below RF
  Improvement vs RF: -60.8%

[41/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.001
  Training: batch_size=32, epochs=80
    Fold 1: RMSE = 95,599.96
    Fold 2: RMSE = 64,683.24
  Result: RMSE = 80,141.60 (+/- 15,458.36) | Below RF
  Improvement vs RF: -87.2%

[42/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.001
  Training: batch_size=32, epochs=80
```

```
    Fold 1: RMSE = 63,510.13
    Fold 2: RMSE = 34,058.79
  Result: RMSE = 48,784.46 (+/- 14,725.67) | Below RF
  Improvement vs RF: -14.0%

[43/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.001
  Training: batch_size=32, epochs=120
    Fold 1: RMSE = 83,639.38
    Fold 2: RMSE = 39,849.36
  Result: RMSE = 61,744.37 (+/- 21,895.01) | Below RF
  Improvement vs RF: -44.2%

[44/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.001
  Training: batch_size=32, epochs=120
    Fold 1: RMSE = 77,548.14
    Fold 2: RMSE = 56,548.12
  Result: RMSE = 67,048.13 (+/- 10,500.01) | Below RF
  Improvement vs RF: -56.6%

[45/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.001
  Training: batch_size=64, epochs=80
    Fold 1: RMSE = 74,424.89
    Fold 2: RMSE = 51,080.38
  Result: RMSE = 62,752.63 (+/- 11,672.25) | Below RF
  Improvement vs RF: -46.6%

[46/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.001
  Training: batch_size=64, epochs=80
    Fold 1: RMSE = 66,618.12
    Fold 2: RMSE = 69,403.97
  Result: RMSE = 68,011.05 (+/- 1,392.92) | Below RF
  Improvement vs RF: -58.9%

[47/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.001
  Training: batch_size=64, epochs=120
    Fold 1: RMSE = 68,942.69
    Fold 2: RMSE = 87,870.40
  Result: RMSE = 78,406.55 (+/- 9,463.86) | Below RF
  Improvement vs RF: -83.2%

[48/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.001
  Training: batch_size=64, epochs=120
```

```
    Fold 1: RMSE = 66,001.20
    Fold 2: RMSE = 50,614.05
  Result: RMSE = 58,307.62 (+/- 7,693.57) | Below RF
  Improvement vs RF: -36.2%

[49/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.0005
  Training: batch_size=32, epochs=80
    Fold 1: RMSE = 72,138.91
    Fold 2: RMSE = 58,787.02
  Result: RMSE = 65,462.97 (+/- 6,675.94) | Below RF
  Improvement vs RF: -52.9%

[50/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.0005
  Training: batch_size=32, epochs=80
    Fold 1: RMSE = 69,894.54
    Fold 2: RMSE = 51,524.00
  Result: RMSE = 60,709.27 (+/- 9,185.27) | Below RF
  Improvement vs RF: -41.8%

[51/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.0005
  Training: batch_size=32, epochs=120
    Fold 1: RMSE = 77,382.78
    Fold 2: RMSE = 50,666.99
  Result: RMSE = 64,024.89 (+/- 13,357.90) | Below RF
  Improvement vs RF: -49.6%

[52/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.0005
  Training: batch_size=32, epochs=120
    Fold 1: RMSE = 67,141.96
    Fold 2: RMSE = 47,017.66
  Result: RMSE = 57,079.81 (+/- 10,062.15) | Below RF
  Improvement vs RF: -33.4%

[53/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.0005
  Training: batch_size=64, epochs=80
    Fold 1: RMSE = 79,202.19
    Fold 2: RMSE = 59,806.30
  Result: RMSE = 69,504.24 (+/- 9,697.94) | Below RF
  Improvement vs RF: -62.4%

[54/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.0005
  Training: batch_size=64, epochs=80
```

```
    Fold 1: RMSE = 70,288.17
    Fold 2: RMSE = 25,186.98
  Result: RMSE = 47,737.58 (+/- 22,550.59) | Below RF
  Improvement vs RF: -11.5%

[55/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.0005
  Training: batch_size=64, epochs=120
    Fold 1: RMSE = 83,134.53
    Fold 2: RMSE = 50,942.90
  Result: RMSE = 67,038.72 (+/- 16,095.82) | Below RF
  Improvement vs RF: -56.6%

[56/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.0005
  Training: batch_size=64, epochs=120
    Fold 1: RMSE = 88,612.13
    Fold 2: RMSE = 62,207.79
  Result: RMSE = 75,409.96 (+/- 13,202.17) | Below RF
  Improvement vs RF: -76.2%

[57/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.0005
  Training: batch_size=32, epochs=80
    Fold 1: RMSE = 79,134.85
    Fold 2: RMSE = 68,997.32
  Result: RMSE = 74,066.09 (+/- 5,068.76) | Below RF
  Improvement vs RF: -73.0%

[58/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.0005
  Training: batch_size=32, epochs=80
    Fold 1: RMSE = 79,350.30
    Fold 2: RMSE = 48,000.12
  Result: RMSE = 63,675.21 (+/- 15,675.09) | Below RF
  Improvement vs RF: -48.8%

[59/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.0005
  Training: batch_size=32, epochs=120
    Fold 1: RMSE = 69,684.34
    Fold 2: RMSE = 61,036.54
  Result: RMSE = 65,360.44 (+/- 4,323.90) | Below RF
  Improvement vs RF: -52.7%

[60/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.0005
  Training: batch_size=32, epochs=120
```

```
    Fold 1: RMSE = 65,768.94
    Fold 2: RMSE = 49,719.18
  Result: RMSE = 57,744.06 (+/- 8,024.88) | Below RF
  Improvement vs RF: -34.9%

[61/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.0005
  Training: batch_size=64, epochs=80
    Fold 1: RMSE = 81,946.02
    Fold 2: RMSE = 58,684.18
  Result: RMSE = 70,315.10 (+/- 11,630.92) | Below RF
  Improvement vs RF: -64.3%

[62/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.0005
  Training: batch_size=64, epochs=80
    Fold 1: RMSE = 72,340.46
    Fold 2: RMSE = 50,801.72
  Result: RMSE = 61,571.09 (+/- 10,769.37) | Below RF
  Improvement vs RF: -43.8%

[63/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.0005
  Training: batch_size=64, epochs=120
    Fold 1: RMSE = 75,464.59
    Fold 2: RMSE = 53,934.94
  Result: RMSE = 64,699.77 (+/- 10,764.83) | Below RF
  Improvement vs RF: -51.2%

[64/128] Testing:
  Architecture: units=64-32-16, dropout=0.1, lr=0.0005
  Training: batch_size=64, epochs=120
    Fold 1: RMSE = 69,947.66
    Fold 2: RMSE = 58,464.69
  Result: RMSE = 64,206.17 (+/- 5,741.48) | Below RF
  Improvement vs RF: -50.0%

[65/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.001
  Training: batch_size=32, epochs=80
    Fold 1: RMSE = 72,058.08
    Fold 2: RMSE = 27,335.80
  Result: RMSE = 49,696.94 (+/- 22,361.14) | Below RF
  Improvement vs RF: -16.1%

[66/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.001
  Training: batch_size=32, epochs=80
```

```
    Fold 1: RMSE = 58,713.04
    Fold 2: RMSE = 29,529.69
  Result: RMSE = 44,121.36 (+/- 14,591.67) | Below RF
  Improvement vs RF: -3.1%
  NEW BEST: RMSE = 44,121.36

[67/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.001
  Training: batch_size=32, epochs=120
    Fold 1: RMSE = 87,319.55
    Fold 2: RMSE = 53,628.20
  Result: RMSE = 70,473.88 (+/- 16,845.68) | Below RF
  Improvement vs RF: -64.6%

[68/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.001
  Training: batch_size=32, epochs=120
    Fold 1: RMSE = 67,889.05
    Fold 2: RMSE = 35,563.65
  Result: RMSE = 51,726.35 (+/- 16,162.70) | Below RF
  Improvement vs RF: -20.8%

[69/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.001
  Training: batch_size=64, epochs=80
    Fold 1: RMSE = 85,572.01
    Fold 2: RMSE = 55,958.73
  Result: RMSE = 70,765.37 (+/- 14,806.64) | Below RF
  Improvement vs RF: -65.3%

[70/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.001
  Training: batch_size=64, epochs=80
    Fold 1: RMSE = 88,762.82
    Fold 2: RMSE = 41,888.58
  Result: RMSE = 65,325.70 (+/- 23,437.12) | Below RF
  Improvement vs RF: -52.6%

[71/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.001
  Training: batch_size=64, epochs=120
    Fold 1: RMSE = 70,861.84
    Fold 2: RMSE = 44,958.36
  Result: RMSE = 57,910.10 (+/- 12,951.74) | Below RF
  Improvement vs RF: -35.3%

[72/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.001
```

```
  Training: batch_size=64, epochs=120
    Fold 1: RMSE = 71,778.43
    Fold 2: RMSE = 42,721.44
  Result: RMSE = 57,249.93 (+/- 14,528.50) | Below RF
  Improvement vs RF: -33.7%

[73/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.001
  Training: batch_size=32, epochs=80
    Fold 1: RMSE = 90,811.74
    Fold 2: RMSE = 52,262.63
  Result: RMSE = 71,537.19 (+/- 19,274.56) | Below RF
  Improvement vs RF: -67.1%

[74/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.001
  Training: batch_size=32, epochs=80
    Fold 1: RMSE = 64,681.05
    Fold 2: RMSE = 67,144.96
  Result: RMSE = 65,913.01 (+/- 1,231.95) | Below RF
  Improvement vs RF: -54.0%

[75/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.001
  Training: batch_size=32, epochs=120
    Fold 1: RMSE = 80,032.23
    Fold 2: RMSE = 46,304.63
  Result: RMSE = 63,168.43 (+/- 16,863.80) | Below RF
  Improvement vs RF: -47.6%

[76/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.001
  Training: batch_size=32, epochs=120
    Fold 1: RMSE = 67,915.79
    Fold 2: RMSE = 28,975.55
  Result: RMSE = 48,445.67 (+/- 19,470.12) | Below RF
  Improvement vs RF: -13.2%

[77/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.001
  Training: batch_size=64, epochs=80
    Fold 1: RMSE = 88,655.54
    Fold 2: RMSE = 45,664.01
  Result: RMSE = 67,159.78 (+/- 21,495.76) | Below RF
  Improvement vs RF: -56.9%

[78/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.001
```

```
  Training: batch_size=64, epochs=80
    Fold 1: RMSE = 87,023.68
    Fold 2: RMSE = 49,202.23
  Result: RMSE = 68,112.95 (+/- 18,910.72) | Below RF
  Improvement vs RF: -59.1%

[79/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.001
  Training: batch_size=64, epochs=120
    Fold 1: RMSE = 71,565.18
    Fold 2: RMSE = 51,235.73
  Result: RMSE = 61,400.46 (+/- 10,164.72) | Below RF
  Improvement vs RF: -43.4%

[80/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.001
  Training: batch_size=64, epochs=120
    Fold 1: RMSE = 68,760.16
    Fold 2: RMSE = 57,015.04
  Result: RMSE = 62,887.60 (+/- 5,872.56) | Below RF
  Improvement vs RF: -46.9%

[81/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.0005
  Training: batch_size=32, epochs=80
    Fold 1: RMSE = 74,612.45
    Fold 2: RMSE = 30,665.44
  Result: RMSE = 52,638.95 (+/- 21,973.50) | Below RF
  Improvement vs RF: -23.0%

[82/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.0005
  Training: batch_size=32, epochs=80
    Fold 1: RMSE = 75,029.99
    Fold 2: RMSE = 43,047.20
  Result: RMSE = 59,038.60 (+/- 15,991.40) | Below RF
  Improvement vs RF: -37.9%

[83/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.0005
  Training: batch_size=32, epochs=120
    Fold 1: RMSE = 88,208.24
    Fold 2: RMSE = 81,127.74
  Result: RMSE = 84,667.99 (+/- 3,540.25) | Below RF
  Improvement vs RF: -97.8%

[84/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.0005
```

```
  Training: batch_size=32, epochs=120
    Fold 1: RMSE = 72,302.45
    Fold 2: RMSE = 62,891.11
  Result: RMSE = 67,596.78 (+/- 4,705.67) | Below RF
  Improvement vs RF: -57.9%

[85/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.0005
  Training: batch_size=64, epochs=80
    Fold 1: RMSE = 86,371.64
    Fold 2: RMSE = 71,812.78
  Result: RMSE = 79,092.21 (+/- 7,279.43) | Below RF
  Improvement vs RF: -84.8%

[86/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.0005
  Training: batch_size=64, epochs=80
    Fold 1: RMSE = 95,846.94
    Fold 2: RMSE = 43,786.63
  Result: RMSE = 69,816.78 (+/- 26,030.16) | Below RF
  Improvement vs RF: -63.1%

[87/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.0005
  Training: batch_size=64, epochs=120
    Fold 1: RMSE = 72,073.62
    Fold 2: RMSE = 58,694.59
  Result: RMSE = 65,384.10 (+/- 6,689.52) | Below RF
  Improvement vs RF: -52.8%

[88/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.0005
  Training: batch_size=64, epochs=120
    Fold 1: RMSE = 86,865.27
    Fold 2: RMSE = 49,450.33
  Result: RMSE = 68,157.80 (+/- 18,707.47) | Below RF
  Improvement vs RF: -59.2%

[89/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.0005
  Training: batch_size=32, epochs=80
    Fold 1: RMSE = 73,375.42
    Fold 2: RMSE = 65,174.03
  Result: RMSE = 69,274.72 (+/- 4,100.70) | Below RF
  Improvement vs RF: -61.8%

[90/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.0005
```

```
  Training: batch_size=32, epochs=80
    Fold 1: RMSE = 62,757.63
    Fold 2: RMSE = 68,197.46
  Result: RMSE = 65,477.55 (+/- 2,719.92) | Below RF
  Improvement vs RF: -53.0%

[91/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.0005
  Training: batch_size=32, epochs=120
    Fold 1: RMSE = 86,552.72
    Fold 2: RMSE = 38,666.52
  Result: RMSE = 62,609.62 (+/- 23,943.10) | Below RF
  Improvement vs RF: -46.3%

[92/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.0005
  Training: batch_size=32, epochs=120
    Fold 1: RMSE = 62,107.36
    Fold 2: RMSE = 52,146.49
  Result: RMSE = 57,126.93 (+/- 4,980.43) | Below RF
  Improvement vs RF: -33.5%

[93/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.0005
  Training: batch_size=64, epochs=80
    Fold 1: RMSE = 110,378.32
    Fold 2: RMSE = 82,033.21
  Result: RMSE = 1,000,000.00 (+/- 0.00) | Below RF

[94/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.0005
  Training: batch_size=64, epochs=80
    Fold 1: RMSE = 94,211.33
    Fold 2: RMSE = 52,465.76
  Result: RMSE = 73,338.55 (+/- 20,872.79) | Below RF
  Improvement vs RF: -71.3%

[95/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.0005
  Training: batch_size=64, epochs=120
    Fold 1: RMSE = 78,752.71
    Fold 2: RMSE = 51,607.85
  Result: RMSE = 65,180.28 (+/- 13,572.43) | Below RF
  Improvement vs RF: -52.3%

[96/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.0005
  Training: batch_size=64, epochs=120
```

```
    Fold 1: RMSE = 80,368.55
    Fold 2: RMSE = 45,695.03
  Result: RMSE = 63,031.79 (+/- 17,336.76) | Below RF
  Improvement vs RF: -47.3%

[97/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.001
  Training: batch_size=32, epochs=80
    Fold 1: RMSE = 75,573.08
    Fold 2: RMSE = 45,993.39
  Result: RMSE = 60,783.24 (+/- 14,789.85) | Below RF
  Improvement vs RF: -42.0%

[98/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.001
  Training: batch_size=32, epochs=80
    Fold 1: RMSE = 77,500.32
    Fold 2: RMSE = 64,804.37
  Result: RMSE = 71,152.34 (+/- 6,347.98) | Below RF
  Improvement vs RF: -66.2%

[99/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.001
  Training: batch_size=32, epochs=120
    Fold 1: RMSE = 74,541.01
    Fold 2: RMSE = 41,251.40
  Result: RMSE = 57,896.21 (+/- 16,644.80) | Below RF
  Improvement vs RF: -35.3%

[100/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.001
  Training: batch_size=32, epochs=120
    Fold 1: RMSE = 70,875.89
    Fold 2: RMSE = 32,551.93
  Result: RMSE = 51,713.91 (+/- 19,161.98) | Below RF
  Improvement vs RF: -20.8%

[101/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.001
  Training: batch_size=64, epochs=80
    Fold 1: RMSE = 91,658.18
    Fold 2: RMSE = 53,477.18
  Result: RMSE = 72,567.68 (+/- 19,090.50) | Below RF
  Improvement vs RF: -69.5%

[102/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.001
  Training: batch_size=64, epochs=80
```

```
    Fold 1: RMSE = 74,148.20
    Fold 2: RMSE = 52,185.51
  Result: RMSE = 63,166.86 (+/- 10,981.35) | Below RF
  Improvement vs RF: -47.6%

[103/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.001
  Training: batch_size=64, epochs=120
    Fold 1: RMSE = 86,990.81
    Fold 2: RMSE = 83,573.73
  Result: RMSE = 85,282.27 (+/- 1,708.54) | Below RF
  Improvement vs RF: -99.2%

[104/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.001
  Training: batch_size=64, epochs=120
    Fold 1: RMSE = 74,792.57
    Fold 2: RMSE = 56,056.91
  Result: RMSE = 65,424.74 (+/- 9,367.83) | Below RF
  Improvement vs RF: -52.8%

[105/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.001
  Training: batch_size=32, epochs=80
    Fold 1: RMSE = 75,401.00
    Fold 2: RMSE = 39,255.88
  Result: RMSE = 57,328.44 (+/- 18,072.56) | Below RF
  Improvement vs RF: -33.9%

[106/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.001
  Training: batch_size=32, epochs=80
    Fold 1: RMSE = 70,194.75
    Fold 2: RMSE = 68,390.52
  Result: RMSE = 69,292.63 (+/- 902.12) | Below RF
  Improvement vs RF: -61.9%

[107/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.001
  Training: batch_size=32, epochs=120
    Fold 1: RMSE = 80,180.10
    Fold 2: RMSE = 56,270.53
  Result: RMSE = 68,225.32 (+/- 11,954.78) | Below RF
  Improvement vs RF: -59.4%

[108/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.001
  Training: batch_size=32, epochs=120
```

```
    Fold 1: RMSE = 72,737.02
    Fold 2: RMSE = 38,487.02
  Result: RMSE = 55,612.02 (+/- 17,125.00) | Below RF
  Improvement vs RF: -29.9%

[109/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.001
  Training: batch_size=64, epochs=80
    Fold 1: RMSE = 100,636.09
    Fold 2: RMSE = 58,307.51
  Result: RMSE = 1,000,000.00 (+/- 0.00) | Below RF

[110/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.001
  Training: batch_size=64, epochs=80
    Fold 1: RMSE = 87,124.48
    Fold 2: RMSE = 58,244.79
  Result: RMSE = 72,684.63 (+/- 14,439.84) | Below RF
  Improvement vs RF: -69.8%

[111/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.001
  Training: batch_size=64, epochs=120
    Fold 1: RMSE = 75,585.28
    Fold 2: RMSE = 31,768.31
  Result: RMSE = 53,676.79 (+/- 21,908.48) | Below RF
  Improvement vs RF: -25.4%

[112/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.001
  Training: batch_size=64, epochs=120
    Fold 1: RMSE = 65,925.32
    Fold 2: RMSE = 29,504.88
  Result: RMSE = 47,715.10 (+/- 18,210.22) | Below RF
  Improvement vs RF: -11.5%

[113/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.0005
  Training: batch_size=32, epochs=80
    Fold 1: RMSE = 76,507.05
    Fold 2: RMSE = 53,368.22
  Result: RMSE = 64,937.64 (+/- 11,569.41) | Below RF
  Improvement vs RF: -51.7%

[114/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.0005
  Training: batch_size=32, epochs=80
    Fold 1: RMSE = 66,628.88
```

```
    Fold 2: RMSE = 59,170.97
  Result: RMSE = 62,899.93 (+/- 3,728.95) | Below RF
  Improvement vs RF: -46.9%

[115/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.0005
  Training: batch_size=32, epochs=120
    Fold 1: RMSE = 97,654.68
    Fold 2: RMSE = 63,742.42
  Result: RMSE = 80,698.55 (+/- 16,956.13) | Below RF
  Improvement vs RF: -88.5%

[116/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.0005
  Training: batch_size=32, epochs=120
    Fold 1: RMSE = 77,767.94
    Fold 2: RMSE = 59,751.74
  Result: RMSE = 68,759.84 (+/- 9,008.10) | Below RF
  Improvement vs RF: -60.6%

[117/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.0005
  Training: batch_size=64, epochs=80
    Fold 1: RMSE = 62,720.88
    Fold 2: RMSE = 76,567.43
  Result: RMSE = 69,644.15 (+/- 6,923.27) | Below RF
  Improvement vs RF: -62.7%

[118/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.0005
  Training: batch_size=64, epochs=80
    Fold 1: RMSE = 87,639.80
    Fold 2: RMSE = 74,888.12
  Result: RMSE = 81,263.96 (+/- 6,375.84) | Below RF
  Improvement vs RF: -89.8%

[119/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.0005
  Training: batch_size=64, epochs=120
    Fold 1: RMSE = 71,850.02
    Fold 2: RMSE = 79,592.54
  Result: RMSE = 75,721.28 (+/- 3,871.26) | Below RF
  Improvement vs RF: -76.9%

[120/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.0005
  Training: batch_size=64, epochs=120
    Fold 1: RMSE = 64,031.66
```

```
    Fold 2: RMSE = 43,818.15
  Result: RMSE = 53,924.90 (+/- 10,106.76) | Below RF
  Improvement vs RF: -26.0%

[121/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.0005
  Training: batch_size=32, epochs=80
    Fold 1: RMSE = 81,697.18
    Fold 2: RMSE = 50,664.84
  Result: RMSE = 66,181.01 (+/- 15,516.17) | Below RF
  Improvement vs RF: -54.6%

[122/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.0005
  Training: batch_size=32, epochs=80
    Fold 1: RMSE = 72,997.32
    Fold 2: RMSE = 27,299.05
  Result: RMSE = 50,148.19 (+/- 22,849.14) | Below RF
  Improvement vs RF: -17.2%

[123/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.0005
  Training: batch_size=32, epochs=120
    Fold 1: RMSE = 76,733.12
    Fold 2: RMSE = 67,821.28
  Result: RMSE = 72,277.20 (+/- 4,455.92) | Below RF
  Improvement vs RF: -68.9%

[124/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.0005
  Training: batch_size=32, epochs=120
    Fold 1: RMSE = 77,571.56
    Fold 2: RMSE = 45,479.13
  Result: RMSE = 61,525.34 (+/- 16,046.21) | Below RF
  Improvement vs RF: -43.7%

[125/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.0005
  Training: batch_size=64, epochs=80
    Fold 1: RMSE = 77,023.00
    Fold 2: RMSE = 43,974.53
  Result: RMSE = 60,498.77 (+/- 16,524.23) | Below RF
  Improvement vs RF: -41.3%

[126/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.0005
  Training: batch_size=64, epochs=80
    Fold 1: RMSE = 77,771.02
```

```
    Fold 2: RMSE = 53,983.97
  Result: RMSE = 65,877.50 (+/- 11,893.52) | Below RF
  Improvement vs RF: -53.9%

[127/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.0005
  Training: batch_size=64, epochs=120
    Fold 1: RMSE = 65,429.61
    Fold 2: RMSE = 73,002.23
  Result: RMSE = 69,215.92 (+/- 3,786.31) | Below RF
  Improvement vs RF: -61.7%

[128/128] Testing:
  Architecture: units=64-32-16, dropout=0.2, lr=0.0005
  Training: batch_size=64, epochs=120
    Fold 1: RMSE = 89,702.63
    Fold 2: RMSE = 36,135.06
  Result: RMSE = 62,918.85 (+/- 26,783.78) | Below RF
  Improvement vs RF: -47.0%

============================================================
LSTM ADVANCED TUNING - FINAL RESULTS
============================================================

LSTM - Best Parameters:
  units1: 64
  units2: 32
  units3: 16
  dropout_rate: 0.2
  recurrent_dropout: 0.0
  learning_rate: 0.001
  l2_reg: 0.0001
  use_batch_norm: True
  batch_size: 32
  epochs: 80
  patience: 15

Best Cross-Validated Performance:
RMSE: 44,121.36
MSE:  2,159,611,716.04
MAE:  30,197.42
Improvement over Random Forest: -3.1%

Top 5 LSTM configurations:
     units1  units2  dropout_rate  learning_rate  batch_size   mean_rmse  \
65       64      32        0.2000         0.0000          32 44,121.3600
34       64      32        0.1000         0.0000          32 44,729.8300
1        64      32        0.1000         0.0000          32 46,160.9000
```

```
111      64      32        0.2000         0.0000          64 47,715.1000
53       64      32        0.1000         0.0000          64 47,737.5800

     improvement_vs_rf
65             -3.0800
34             -4.5000
1              -7.8400
111           -11.4700
53            -11.5300

Training final LSTM model with validation…
Training with 80 epochs and 20% validation split…
Epoch 1/80
37/37             11s 57ms/step -
loss: 10.9351 - mae: 11.4071 - val_loss: 10.8086 - val_mae: 11.2806 -
learning_rate: 0.0010
Epoch 2/80
37/37             1s 22ms/step -
loss: 7.8336 - mae: 8.3045 - val_loss: 5.7536 - val_mae: 6.2248 - learning_rate:
0.0010
Epoch 3/80
37/37             1s 21ms/step -
loss: 2.2578 - mae: 2.6900 - val_loss: 1.1571 - val_mae: 1.6277 - learning_rate:
0.0010
Epoch 4/80
37/37             1s 22ms/step -
loss: 1.5598 - mae: 1.9756 - val_loss: 0.6091 - val_mae: 1.0711 - learning_rate:
0.0010
Epoch 5/80
37/37             1s 21ms/step -
loss: 1.4972 - mae: 1.9178 - val_loss: 0.4529 - val_mae: 0.8974 - learning_rate:
0.0010
Epoch 6/80
37/37             1s 21ms/step -
loss: 1.4314 - mae: 1.8420 - val_loss: 0.2776 - val_mae: 0.6594 - learning_rate:
0.0010
Epoch 7/80
37/37             1s 21ms/step -
loss: 1.2734 - mae: 1.6826 - val_loss: 0.1288 - val_mae: 0.4220 - learning_rate:
0.0010
Epoch 8/80
37/37             1s 21ms/step -
loss: 1.2445 - mae: 1.6488 - val_loss: 0.1070 - val_mae: 0.3081 - learning_rate:
0.0010
Epoch 9/80
37/37             1s 20ms/step -
loss: 1.2396 - mae: 1.6517 - val_loss: 0.2283 - val_mae: 0.4629 - learning_rate:
0.0010
```

```
Epoch 10/80
37/37            1s 20ms/step -
loss: 1.1895 - mae: 1.5963 - val_loss: 0.2664 - val_mae: 0.6326 - learning_rate:
0.0010
Epoch 11/80
37/37            1s 20ms/step -
loss: 1.1521 - mae: 1.5560 - val_loss: 0.0879 - val_mae: 0.2335 - learning_rate:
0.0010
Epoch 12/80
37/37            1s 22ms/step -
loss: 1.1727 - mae: 1.5786 - val_loss: 0.2243 - val_mae: 0.4723 - learning_rate:
0.0010
Epoch 13/80
37/37            1s 21ms/step -
loss: 1.1176 - mae: 1.5224 - val_loss: 0.1586 - val_mae: 0.4893 - learning_rate:
0.0010
Epoch 14/80
37/37            1s 21ms/step -
loss: 1.1075 - mae: 1.5084 - val_loss: 0.2505 - val_mae: 0.6365 - learning_rate:
0.0010
Epoch 15/80
37/37            1s 20ms/step -
loss: 1.1367 - mae: 1.5412 - val_loss: 0.0664 - val_mae: 0.2085 - learning_rate:
0.0010
Epoch 16/80
37/37            1s 21ms/step -
loss: 1.0650 - mae: 1.4640 - val_loss: 0.1307 - val_mae: 0.4303 - learning_rate:
0.0010
Epoch 17/80
37/37            1s 21ms/step -
loss: 1.0708 - mae: 1.4722 - val_loss: 0.0775 - val_mae: 0.2812 - learning_rate:
0.0010
Epoch 18/80
37/37            1s 21ms/step -
loss: 1.0925 - mae: 1.4976 - val_loss: 0.2826 - val_mae: 0.6710 - learning_rate:
0.0010
Epoch 19/80
37/37            1s 20ms/step -
loss: 0.9845 - mae: 1.3738 - val_loss: 0.1542 - val_mae: 0.4755 - learning_rate:
0.0010
Epoch 20/80
37/37            1s 21ms/step -
loss: 1.0611 - mae: 1.4614 - val_loss: 0.1552 - val_mae: 0.4705 - learning_rate:
0.0010
Epoch 21/80
37/37            1s 20ms/step -
loss: 0.9974 - mae: 1.3973 - val_loss: 0.1975 - val_mae: 0.5590 - learning_rate:
0.0010
```

```
Epoch 22/80
37/37             1s 21ms/step -
loss: 1.0292 - mae: 1.4247 - val_loss: 0.1202 - val_mae: 0.3786 - learning_rate:
0.0010
Epoch 23/80
37/37             1s 21ms/step -
loss: 1.0308 - mae: 1.4306 - val_loss: 0.1328 - val_mae: 0.3723 - learning_rate:
0.0010
Epoch 24/80
37/37             1s 20ms/step -
loss: 1.0209 - mae: 1.4168 - val_loss: 0.2339 - val_mae: 0.6076 - learning_rate:
5.0000e-04
Epoch 25/80
37/37             1s 21ms/step -
loss: 0.9648 - mae: 1.3628 - val_loss: 0.2443 - val_mae: 0.6301 - learning_rate:
5.0000e-04
Epoch 26/80
37/37             1s 21ms/step -
loss: 1.0040 - mae: 1.3965 - val_loss: 0.1137 - val_mae: 0.3776 - learning_rate:
5.0000e-04
Epoch 27/80
37/37             1s 22ms/step -
loss: 0.9693 - mae: 1.3632 - val_loss: 0.2360 - val_mae: 0.5977 - learning_rate:
5.0000e-04
Epoch 28/80
37/37             1s 20ms/step -
loss: 1.0070 - mae: 1.4031 - val_loss: 0.1715 - val_mae: 0.5131 - learning_rate:
5.0000e-04
Epoch 29/80
37/37             1s 21ms/step -
loss: 0.9650 - mae: 1.3622 - val_loss: 0.1221 - val_mae: 0.4159 - learning_rate:
5.0000e-04
Epoch 30/80
37/37             1s 21ms/step -
loss: 0.9887 - mae: 1.3790 - val_loss: 0.1267 - val_mae: 0.4301 - learning_rate:
5.0000e-04
Final LSTM model trained successfully!

================================================
LSTM PERFORMANCE COMPARISON
================================================
Full Dataset Performance:
RMSE: 67,591.62
MSE:  4,568,627,521.58
MAE:  45,917.93

Validation Set Performance:
RMSE: 45,164.79
```

```
MSE:  2,039,858,492.76
MAE:  33,922.93

Using Validation Set Performance (less overfitting)

============================================================
LSTM FINAL PERFORMANCE (Best RMSE)
============================================================
Source: Validation Set
RMSE: 45,164.79
MSE:  2,039,858,492.76
MAE:  33,922.93

============================================================
COMPARISON WITH RANDOM FOREST
============================================================
LSTM Final RMSE:  45,164.79
Random Forest RMSE: 42,804.37
Difference: 2,360.42
 LSTM trails Random Forest by -5.5%

LSTM advanced tuning completed!
Total parameter combinations tested: 128

LSTM Performance Metrics Available for Comparison:
LSTM_FINAL_RMSE = 45,164.79
LSTM_FINAL_MSE  = 2,039,858,492.76
LSTM_FINAL_MAE  = 33,922.93
```

0.1.7 7. Compare two models

[ ]:
```python
# COMPARE MODELS PERFORMANCE

# Extract Random Forest performance metrics from Step 5
rf_best_result = results_df.iloc[0]
rf_rmse = rf_best_result['mean_rmse']
rf_rmse_std = rf_best_result['std_rmse']
rf_mse = rf_best_result['mean_mse']
rf_mae = rf_best_result['mean_mae']
rf_r2 = rf_best_result['mean_r2']
rf_r2_std = rf_best_result['std_r2']

# Extract LSTM performance metrics from Step 6
lstm_rmse = LSTM_FINAL_RMSE
lstm_mse = LSTM_FINAL_MSE
lstm_mae = LSTM_FINAL_MAE
lstm_r2 = best_lstm_r2
```

```
lstm_r2_std = best_lstm_r2_std

# For LSTM standard deviation, use the best configuration's std from␣
 ↪cross-validation
if 'best_lstm_std' in locals():
    lstm_rmse_std = best_lstm_rmse_std
else:
    # If not available, estimate based on cross-validation results
    if len(lstm_results_df) > 0:
        lstm_rmse_std = lstm_results_df.iloc[0]['std_rmse']
    else:
        lstm_rmse_std = 0  # Default if no std available

# Calculate MSE standard deviations (approximate based on RMSE std)
rf_mse_std = 2 * rf_rmse * rf_rmse_std if rf_rmse_std > 0 else 0
lstm_mse_std = 2 * lstm_rmse * lstm_rmse_std if lstm_rmse_std > 0 else 0

# Create comparison table
print("\n" + "="*50)
print("FINAL SUMMARY TABLE")
print("="*50)

# Format numbers with proper formatting
def format_metric(value, std=0, is_currency=False, is_squared=False):
    if is_currency:
        if is_squared:
            value_str = f"{value:,.0f}"
            std_str = f"{std:,.0f}" if std > 0 else "0"
        else:
            value_str = f"{value:,.2f}"
            std_str = f"{std:,.2f}" if std > 0 else "0.00"
    else:
        value_str = f"{value:.4f}"
        std_str = f"{std:.4f}" if std > 0 else "0.0000"

    if std > 0:
        return f"{value_str} ± {std_str}"
    else:
        return value_str

# Create the comparison table
comparison_data = {
    'Metric': ['RMSE ($)', 'MSE ($²)', 'MAE ($)', 'R²'],
    'Random Forest': [
        format_metric(rf_rmse, rf_rmse_std, is_currency=True, is_squared=False),
        format_metric(rf_mse, rf_mse_std, is_currency=True, is_squared=True),
        format_metric(rf_mae, rf_rmse_std, is_currency=True, is_squared=False),
```

```
        format_metric(rf_r2, rf_r2_std, is_currency=False, is_squared=False)
    ],
    'LSTM': [
        format_metric(lstm_rmse, lstm_rmse_std, is_currency=True,␣
 ↪is_squared=False),
        format_metric(lstm_mse, lstm_mse_std, is_currency=True,␣
 ↪is_squared=True),
        format_metric(lstm_mae, lstm_rmse_std, is_currency=True,␣
 ↪is_squared=False),
        format_metric(lstm_r2, lstm_r2_std, is_currency=False, is_squared=False)
    ]
}

comparison_df = pd.DataFrame(comparison_data)
print(comparison_df.to_string(index=False))

# Create DataFrame for nice formatting
comparison_df = pd.DataFrame(comparison_data)

# Display the result
print("\n" + "-" * 80)
print(f"{'Metric':<15} {'Random Forest':<35} {'LSTM':<35}")
print("-" * 80)
for i, row in comparison_df.iterrows():
    print(f"{row['Metric']:<15} {row['Random Forest']:<35} {row['LSTM']:<35}")
print("-" * 80)

# Also print a simple numeric comparison without formatting
print("\n" + "="*60)
print("NUMERIC COMPARISON (without formatting)")
print("="*60)
print(f"Random Forest - RMSE: {rf_rmse:,.2f}, MSE: {rf_mse:,.0f}, MAE: {rf_mae:
 ↪,.2f}, R²: {rf_r2:.4f}")
print(f"LSTM          - RMSE: {lstm_rmse:,.2f}, MSE: {lstm_mse:,.0f}, MAE:␣
 ↪{lstm_mae:,.2f}, R²: {lstm_r2:.4f}")
```

```
================================================
FINAL SUMMARY TABLE
================================================

--------------------------------------------------------------------------------
Metric          Random Forest                   LSTM
--------------------------------------------------------------------------------
RMSE ($)        42,804.37 ± 3,187.63            45,164.79 ± 14,591.67
MSE ($²)        1,842,375,113 ± 272,888,599     2,039,858,493 ± 1,318,059,850
MAE ($)         29,365.84 ± 3,187.63            33,922.93 ± 14,591.67
```
